Contents

1 Introduction and Learning Objectives **1**

2 Logic **5**
2.1 Propositional Logic 7
2.1.1 Propositions 7
2.1.2 Logical operators 8
2.1.3 Precedence rules 9
2.1.4 Logical equivalence 10
2.1.5 More logical operators 11
2.1.6 Implications in English 12
2.1.7 More forms of implication 14
2.1.8 Exclusive or 15
2.1.9 Universal operators 15
2.1.10 Classifying propositions 16
2.2 Boolean Algebra 18
2.2.1 Basics of Boolean Algebra 20
2.2.2 Substitution laws 21
2.2.3 Simplifications 23
2.2.4 More rules of Boolean algebra 24
2.3 Application: Logic Circuits 28
2.3.1 Logic gates * 29
2.3.2 Combining gates to create circuits * 30
2.3.3 From circuits to propositions * 31
2.3.4 Disjunctive Normal Form 32
2.3.5 Binary addition * 37
2.4 Predicate Logic 40
2.4.1 Predicates 40
2.4.2 Quantifiers 42
2.4.3 Operators 44
2.4.4 Tarski's world and formal structures 47
2.4.5 Logical equivalence 48

2.5 Deduction . . . 52
2.5.1 Arguments . . . 53
2.5.2 Valid arguments and proofs . . . 55
2.5.3 Proofs in predicate logic . . . 58

3 Proof **61**
3.1 A Little Historical Background . . . 62
3.2 Mathematical Proof . . . 63
3.2.1 How to write a proof . . . 66
3.2.2 Some terminology . . . 69
3.2.3 Examples . . . 70
3.3 Proof by Contradiction . . . 74
3.4 Mathematical Induction . . . 77
3.4.1 How to write a proof by induction . . . 78
3.4.2 Examples . . . 79
3.4.3 More examples . . . 81
3.5 Strong Mathematical Induction . . . 83
3.6 Application: Recursion and Induction . . . 86
3.6.1 Recursive factorials . . . 86
3.6.2 Towers of Hanoi * . . . 88
3.7 Recursive Definitions . . . 91
3.8 Trees . . . 93
3.8.1 Nomenclature of trees . . . 94
3.8.2 An application of trees . . . 95
3.8.3 Binary trees in Java * . . . 96
3.9 Invariants . . . 100

4 Sets, Functions, and Relations **103**
4.1 Basic Concepts . . . 103
4.1.1 Elements of sets . . . 105
4.1.2 Set-builder notation . . . 106
4.1.3 Operations on sets . . . 108
4.1.4 Visualising sets . . . 109
4.1.5 Sets of sets . . . 111
4.1.6 Ordered collections: Tuples . . . 113
4.1.7 One more set operation: Cartesian product . . . 114
4.1.8 Mathematical induction revisited . . . 114
4.1.9 Structural induction . . . 115
4.1.10 Revisiting trees . . . 118
4.2 The Boolean Algebra of Sets . . . 121
4.2.1 Set complement . . . 122
4.2.2 Link between logic and set theory . . . 125
4.3 Application: Graphs . . . 130

4.3.1 Graph nomenclature . . . 130
4.3.2 An application of graphs: Task ordering . . . 131
4.4 Application: Programming with Sets * . . . 132
4.4.1 Representing sets . . . 132
4.4.2 Computing with sets . . . 135
4.5 Functions . . . 139
4.5.1 Formalising the notion of functions . . . 140
4.5.2 Operations on functions . . . 141
4.5.3 Properties of functions . . . 143
4.5.4 Functions on trees . . . 145
4.5.5 Functions on graphs . . . 146
4.5.6 First-class objects . . . 147
4.6 Application: Programming with Functions * . . . 149
4.6.1 Functions as first-class objects . . . 151
4.7 Counting Past Infinity . . . 154
4.7.1 Cardinality . . . 155
4.7.2 Counting to infinity . . . 158
4.7.3 Uncountable sets * . . . 160
4.7.4 A final note on infinities * . . . 163
4.8 Relations . . . 165
4.8.1 Properties of relations . . . 166
4.8.2 Equivalence relations . . . 168
4.9 Application: Relational Databases * . . . 173

5 Looking Beyond * 179

Selected Solutions 183

Further Reading 211

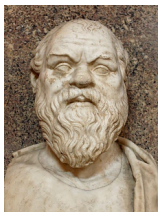
Index 213

Chapter 2

Logic

In a sense, we know a lot more than we realise, because everything that we know has consequences—*logical* consequences—that follow automatically. If you know that all humans are mortal, and you know that Socrates is human, then in a sense you know that Socrates is mortal, whether or not you have ever considered or wanted to consider that fact. This is an example of ***logical deduction***: from the ***premises*** that "All humans are mortal" and "Socrates is human", the ***conclusion*** that "Socrates is mortal" can be deduced by logic.





Socrates was a Greek philosopher who suffered a most unfortunate fate. In one of the most famous mathematical arguments—the one outlined above—he is the man that is destined to die. History has since taught us we were right as Socrates died after a long life (71 years), sentenced to death for corrupting the minds of the youth of Athens. His disciple Plato wrote many Socratic dialogues which give an insight into the philosophy of Socrates, often summarised as: "I know that I know nothing". The last words of this mortal man were (according to Plato): "Crito, we owe a cock to Asclepius: pay it and do not neglect it." The fate of the chicken is unknown...

Source: `en.wikipedia.org/wiki/Socrates`.

2

Logical deduction is a kind of computation. By applying rules of logic to a given set of premises, conclusions that follow from those premises can be generated automatically. This computational process could for instance be carried out by a computer. Once you know the premises, or are willing to accept them for the sake of argument, you are forced *by logic* to accept the conclusions. Still, to say that you 'know' those conclusions would be misleading. The problem is that there are too many of them (infinitely many), and, in general, most of them are not particularly interesting. Until you have actually made the deduction, you don't *really* know the conclusion, and knowing which of the possible chains of deduction to follow is not easy. The *art* of logic is to find an interesting conclusion and a chain of logical deductions that leads from the premises to that conclusion. Checking that the deductions are valid is the mechanical, computational side of logic.



Later in *Reasoning & Logic*, you will see some automated computational techniques that can help us checking the deductions. We don't cover these in this edition of this book. In fact, there are automated *proof assistants* that can even help us finding interesting conclusions. One of the more famous is called ***Coq***, a name perhaps inspired by Socrates's chicken.

This chapter is mostly about the mechanics of logic. We will investigate logic as a branch of mathematics, with its own symbols, formulas and rules of computation. Your objective is to learn the rules of logic, to understand why they are valid, and to develop skill in applying them. As with any branch of mathematics, there is a certain beauty to the symbols and formulas themselves. But it is the applications that bring the subject to life for most people. We will, of course, cover some applications as we go along. In a sense, though, the real applications of logic include much of computer science and of mathematics itself.

Among the fundamental elements of thought, and therefore of logic, are propositions. A ***proposition*** is a statement that has a truth value: it is either true or false. "Delft is a city" and "$2 + 2 = 42$" are propositions. In the first part of this chapter, we will study ***propositional logic***, which takes propositions and considers how they can be combined and manipulated. This branch of logic has surprising application to the design of the electronic circuits that make up computers. This ties closely to the digital and boolean logic you will study in your course *Computer Organisation*.

Logic gets more interesting when we consider the internal structure of propositions. In English, a proposition is expressed as a sentence, and, as you know from studying grammar, sentences have parts. A simple sentence like "Delft is a city" has a ***subject*** and a ***predicate***. The sentence says some-

thing about its subject. The subject of "Delft is a city" is Delft. The sentence says something about Delft. The *something* that the sentence says about its subject is the predicate. In the example, the predicate is the phrase 'is a city'. Once we start working with predicates, we can create propositions using ***quantifiers*** like 'all', 'some' and 'no'. For example, working with the predicate 'has a university' we can move from simple propositions like "Delft has a university" to "All cities have a university" or to "No city has a university" or to the rather more realistic "Some cities have a university".

Logical deduction usually deals with quantified statements, as shown by the basic example of human mortality with which we began this chapter. Logical deduction will be a major topic of this chapter; and under the name of ***proof***, it will be the topic of the next chapter and a major tool for the rest of this book and indeed your computer science degree programme.

2.1 Propositional Logic

We humans use a ***natural language*** when we speak, such as Dutch, English or Flemish. Natural languages are ambiguous and often vague. To start modelling them we first consider ***propositional logic***. This form of logic is arguably the easiest to work with, but also has limited expressive power. However even with this form we can already encapsulate many arguments and power a number of applications, for instance digital logic in chip design.

2.1.1 Propositions

A proposition is a statement which is either true or false. In propositional logic, we reason only about propositions and see what we can do with them. Since this is mathematics, we need to be able to talk about propositions without saying which particular propositions we are talking about, so we use symbolic names to represent them. We will always use lowercase letters such as p, q and r to represent propositions. A letter used in this way is called a ***propositional variable***. Remember that when we say something like "Let p be a proposition", we mean "For the rest of this discussion, let the symbol p stand for some particular statement, which is either true or false (although we're not at the moment making any assumption about which it is)." The discussion has ***mathematical generality*** in that p can represent any statement, and the discussion will be valid no matter which statement it represents.



Propositional variables are a little bit like variables in a programming language such as Java. A basic Java variable such as `int x` can take any integer value. There is 'a little bit' of similarity between the two notions of variables—don't take the analogy too far at this point in your learning!

2

2.1.2 Logical operators

What we do with propositions is combine them with ***logical operators***, also referred to as ***logical connectives***. A logical operator can be applied to one or more propositions to produce a new proposition. The truth value of the new proposition is completely determined by the operator and by the truth values of the propositions to which it is applied.[1] In English, logical operators are represented by words such as 'and', 'or' and 'not'. For example, the proposition "I wanted to leave and I left" is formed from two simpler propositions joined by the word 'and'. Adding the word 'not' to the proposition "I left" gives "I did not leave" (after a bit of necessary grammatical adjustment).

But English is a little too rich for mathematical logic. When you read the sentence "I wanted to leave and I left", you probably see a connotation of causality: I left *because* I wanted to leave. This implication does not follow from the logical combination of the truth values of the two propositions "I wanted to leave" and "I left". Or consider the proposition "I wanted to leave but I did not leave". Here, the word 'but' has the same *logical* meaning as the word 'and', but the connotation is very different. So, in mathematical logic, we use *symbols* to represent logical operators. These symbols do not carry any connotation beyond their defined logical meaning. The logical operators corresponding to the English words 'and', 'or' and 'not' are $\wedge$, $\vee$ and $\neg$.[2]

Definition 2.1. Let p and q be propositions. Then $p \vee q$, $p \wedge q$, and $\neg p$ are propositions, whose truth values are given by the rules:

- $p \wedge q$ is true when both p is true and q is true, and in no other case
- $p \vee q$ is true when either p is true, or q is true, or both p and q are true, and in no other case

[1] It is not always true that the truth value of a sentence can be determined from the truth values of its component parts. For example, if p is a proposition, then 'Johan Cruyff believes p' is also a proposition, so 'Cruyff believes' is some kind of operator. However, it does not count as a *logical* operator because just from knowing whether or not p is true, we get no information at all about whether 'Johan Cruyff believes p' is true.

[2] Other textbooks might use different notations to represent a negation. For instance a bar over the variable ($\bar{x}$) or a $\sim$ symbol ($\sim x$). In Boolean algebra (and thus in your *Computer Organisation* course) you will also often find the $+$ symbol to represent an 'or' and a $\cdot$ (dot) symbol to represent an 'and'.

- $\neg p$ is true when p is false, and in no other case

The operators $\wedge$, $\vee$ and $\neg$ are referred to as ***conjunction***, ***disjunction*** and ***negation***, respectively. (Note that $p \wedge q$ is read as 'p and q', $p \vee q$ is read as 'p or q', and $\neg p$ is read as 'not p'.)



Consider the statement "I am a CSE student or I am not a TPM student." Taking p to mean "I am a CSE student" and q to mean "I am a TPM student", you can write this as $p \vee \neg q$.

2.1.3 Precedence rules

These operators can be used in more complicated expressions, such as $p \wedge \neg q$ or $(p \vee q) \wedge (q \vee r)$. A proposition made up of simpler propositions and logical operators is called a ***compound proposition***. Just like in mathematics, parentheses can be used in compound expressions to indicate the order in which the operators are to be evaluated. In the absence of parentheses, the order of evaluation is determined by ***precedence rules***. For the logical operators defined above, the rules are that $\neg$ has higher precedence than $\wedge$, and $\wedge$ has precedence over $\vee$. This means that in the absence of parentheses, any $\neg$ operators are evaluated first, followed by any $\wedge$ operators, followed by any $\vee$ operators.

For example, the expression $\neg p \vee q \wedge r$ is equivalent to the expression $(\neg p) \vee (q \wedge r)$, while $p \vee q \wedge q \vee r$ is equivalent to $p \vee (q \wedge q) \vee r$.

This still leaves open the question of which of the $\wedge$ operators in the expression $p \wedge q \wedge r$ is evaluated first. This is settled by the following rule: When several operators of equal precedence occur in the absence of parentheses, they are evaluated from left to right. Thus, the expression $p \wedge q \wedge r$ is equivalent to $(p \wedge q) \wedge r$ rather than to $p \wedge (q \wedge r)$. In this particular case, as a matter of fact, it doesn't really matter which $\wedge$ operator is evaluated first, since the two compound propositions $(p \wedge q) \wedge r$ and $p \wedge (q \wedge r)$ always have the same value, no matter what logical values the component propositions p, q, and r have. We say that $\wedge$ is an ***associative*** operation. We'll see more about associativity and other properties of operations in the next section.



In practice however you should **always** add parentheses in places where ambiguity may arise. In fact some textbooks even add them to single operators as well, e.g., writing $(p \wedge q)$ instead of $p \wedge q$. Although for this course we do not require them around single operators, we should never need the precedence rules outlined above. Your parentheses should make clear the order of operations!

p	q	r	$p \wedge q$	$q \wedge r$	$(p \wedge q) \wedge r$	$p \wedge (q \wedge r)$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

Figure 2.1: *A truth table that demonstrates the logical equivalence of* $(p \wedge q) \wedge r$ *and* $p \wedge (q \wedge r)$*. The fact that the last two columns of this table are identical shows that these two expressions have the same value for all eight possible combinations of values of* p*,* q*, and* r*.*

Every compound proposition has a ***main connective***. The main connective is the connective that is evaluated last, according to the precedence rules and parentheses. There should be no ambiguity over which is the main connective in a compound proposition.

2.1.4 Logical equivalence

Suppose we want to verify that, in fact, $(p \wedge q) \wedge r$ and $p \wedge (q \wedge r)$ do always have the same value. To do so, we have to consider all possible combinations of values of p, q, and r, and check that for all such combinations, the two compound expressions do indeed have the same value. It is convenient to organize this computation into a truth table. A ***truth table*** is a table that shows the value of one or more compound propositions for each possible combination of values of the propositional variables that they contain. We call each such combination a ***situation***. Figure 2.1 is a truth table that compares the value of $(p \wedge q) \wedge r$ to the value of $p \wedge (q \wedge r)$ for all possible values of p, q, and r. There are eight rows in the table because there are exactly eight different ways in which truth values can be assigned to p, q, and r.[3] In this table, we see that the last two columns, representing the values of $(p \wedge q) \wedge r$ and $p \wedge (q \wedge r)$, are identical.

[3] In general, if there are n variables, then there are 2^n different ways to assign truth values to the variables, i.e., 2^n situations. This might become clear to you if you try to come up with a scheme for systematically listing all possible sets of values. As this should not satisfy you, you'll find a rigorous proof of the fact later in this chapter.



I discuss the creation of truth tables for statements written in propositional logic in more detail in one of the pencasts of this course: youtu.be/oua_nvpFECQ.



In another pencast of this course, we discuss how you should formulate your answer when using truth tables to test for equivalence:
youtu.be/sWu0fUu7s5c.



You can write the rows in a truth table in any order you like. We suggest you write them in a sorted order, as in Table 2.1. This helps you to be systematic in writing out the table. It also helps us to provide feedback on your answers!

More generally, we say that two compound propositions are ***logically equivalent*** if they always have the same value, no matter what truth values are assigned to the propositional variables that they contain. If the number of propositional variables is small, it is easy to use a truth table to check whether or not two propositions are logically equivalent.



When writing a piece of code you will often have your code make decisions. For instance in a bit of Java code—such as in your *Object-Oriented Programming* course—you might encounter an if-statement to check if the user has inputted the right type of data. Since the input you expect can be rather difficult, the if-statement is a complex combination of many simple checked chained together by `&&`'s and `||`'s. After taking a look at the code, you believe it can be simplified to a much smaller expression. Using a truth table you can prove that your simplified version is equivalent to the original.

2.1.5 More logical operators

There are other logical operators besides $\wedge$, $\vee$, and $\neg$. We will consider the ***conditional operator***, $\rightarrow$, the ***biconditional operator***, $\leftrightarrow$, and the ***exclusive***

or operator, $\oplus$.[4] These operators can be completely defined by a truth table that shows their values for the four possible combinations of truth values of p and q.

2

Definition 2.2. For any propositions p and q, we define the propositions $p \to q$, $p \leftrightarrow q$, and $p \oplus q$ according to the truth table:

p	q	$p \to q$	$p \leftrightarrow q$	$p \oplus q$
0	0	1	1	0
0	1	1	0	1
1	0	0	0	1
1	1	1	1	0



When these operators are used in expressions, in the absence of parentheses to indicate order of evaluation, we use the following precedence rules: The exclusive or operator, $\oplus$, has the same precedence as $\vee$. The conditional operator, $\to$, has lower precedence than $\wedge$, $\vee$, $\neg$, and $\oplus$, and is therefore evaluated after them. Finally, the biconditional operator, $\leftrightarrow$, has the lowest precedence and is therefore evaluated last. For example, the expression $p \to q \wedge r \leftrightarrow \neg p \oplus s$ is evaluated as if it were written $(p \to (q \wedge r)) \leftrightarrow ((\neg p) \oplus s)$. But again you should always include the parentheses!

In order to work effectively with the logical operators, you need to know more about their meaning and how they relate to ordinary English expressions. To that end we first consider the conditional operator in more detail in the next section.

2.1.6 Implications in English

The proposition $p \to q$ is called an ***implication*** or a ***conditional***. It is usually read as 'p implies q'. In such an implication p and q also get special names of their own. p is called the ***hypothesis*** or ***antecedent*** and q is called the ***conclusion*** or ***consequent***.

Furthermore we say that if the implication $p \to q$ holds, then p is ***sufficient*** for q. That is if p is true that is sufficient to also make q true. Conversely

[4] Note that the symbols for these operations also differ from textbook to textbook. While $\to$ is fairly standard, $\leftrightarrow$ is sometimes represented by $\equiv$ or $\Leftrightarrow$. There is even less standardization of the exclusive or operator, but that operator is generally not so important as the others.

we say that q is ***necessary*** for p. Without q being true, it is impossible for p to be true. That is if q is false, then p also has to be false.

In English, $p \to q$ is often expressed as 'if p then q'. For example, if p represents the proposition "Karel Luyben is Rector Magnificus of TU Delft" and q represents "Prometheus is blessed by the gods", then $p \to q$ could be expressed in English as "If Karel Luyben is Rector Magnificus of TU Delft, then Prometheus is blessed by the gods." In this example, p is false and q is also false. Checking the definition of $p \to q$, we see that $p \to q$ is a true statement. Most people would agree with this, even though it is not immediately obvious.



The letter 'T' in the TUDelft logo bears a stylized flame on top, referring to the flame that Prometheus brought from Mount Olympus to the people, against the will of Zeus. Because of this, Prometheus is sometimes considered as the first engineer, and he is an important symbol for the university. His bronze statue stands in the Mekelpark at the centre of campus.



Source: `en.wikipedia.org/wiki/Delft_University_of_Technology`.
Image: `weblog.library.tudelft.nl/2016/01/04/english-prometheus-is-back/`.

It is worth looking at a similar example in more detail. Suppose that I assert that "If Feyenoord is a great team, then I'm the King of the Netherlands". This statement has the form $m \to k$ where m is the proposition "Feyenoord is a great team" and k is the proposition "I'm the king of the Netherlands". Now, demonstrably I am not the king of the Netherlands, so k is false. Since k is false, the only way for $m \to k$ to be true is for m to be false as well. (Check the definition of $\to$ in the table, if you are not convinced!) So, by asserting $m \to k$, I am really asserting that the Feyenoord is *not* a great team.

Or consider the statement, "If the party is on Tuesday, then I'll be there."

What am I trying to say if I assert this statement? I am asserting that $p \to q$ is true, where p represents "The party is on Tuesday" and q represents "I will be at the party". Suppose that p is true, that is, the party does in fact take place on Tuesday. Checking the definition of $\to$, we see that in the only case where p is true and $p \to q$ is true, q is also true. So from the truth of "If the party is on Tuesday, then I will be at the party" and "The party is in fact on Tuesday", you can deduce that "I will be at the party" is also true. But suppose, on the other hand, that the party is actually on Wednesday. Then p is false. When p is false and $p \to q$ is true, the definition of $p \to q$ allows q to be either true or false. So, in this case, you can't make any deduction about whether or not I will be at the party. The statement "If the party is on Tuesday, then I'll be there" doesn't assert anything about what will happen if the party is on some other day than Tuesday.

2.1.7 More forms of implication

The implication $\neg q \to \neg p$ is called the ***contrapositive*** of $p \to q$. An implication is logically equivalent to its contrapositive. The contrapositive of "If this is Tuesday, then we are in Belgium" is "If we aren't in Belgium, then this isn't Tuesday". These two sentences assert exactly the same thing.

Note that $p \to q$ is *not* logically equivalent to $q \to p$. The implication $q \to p$ is called the ***converse*** of $p \to q$. The converse of "If this is Tuesday, then we are in Belgium" is "If we are in Belgium, then this is Tuesday". Note that it is possible for either one of these statements to be true while the other is false. In English, we might express the fact that both statements are true by saying "If this is Tuesday, then we are in Belgium, *and conversely*". In logic, this would be expressed with a proposition of the form $(p \to q) \wedge (q \to p)$.

Similarly $p \to q$ is *not* logically equivalent to $\neg p \to \neg q$. The implication $\neg p \to \neg q$ is called the ***inverse*** of $p \to q$. Although this mistake is commonly made in English, for instance people often assume that when I say: "If it is morning, I drink some coffee", I also mean that when it is not morning I do not drink coffee. But my original statement does not tell you anything about what I do when it is not morning.

The ***biconditional operator*** is closely related to the conditional operator. In fact, $p \leftrightarrow q$ is logically equivalent to $(p \to q) \wedge (q \to p)$. The proposition $p \leftrightarrow q$ is usually read as 'p if and only if q'. (The 'p if q' part represents $q \to p$, while 'p only if q' is another way of asserting that $p \to q$.) It could also be expressed as 'if p then q, and conversely'. Occasionally in English, 'if... then' is used when what is really meant is 'if and only if'. For example, if a parent tells a child, "If you are good, Sinterklaas will bring you toys", the parent probably really means to say "Sinterklaas will bring you toys if and only if you are good". (The parent would probably not respond well to

the child's perfectly logical plea "But you never said what would happen if I wasn't good!")

2.1.8 Exclusive or

Finally, we turn to the exclusive or operator. The English word 'or' is actually somewhat ambiguous. The two operators $\oplus$ and $\vee$ express the two possible meanings of this word. The proposition $p \vee q$ can be expressed unambiguously as "p or q, *or both*", while $p \oplus q$ stands for "p or q, *but not both*". If a menu says that you can choose soup or salad, it doesn't mean that you can have both. In this case, 'or' is an exclusive or. On the other hand, in "You are at risk of heart disease if you smoke or drink", the or is inclusive since you certainly don't get off the hook if you both smoke and drink. In theoretical computer science and mathematics, the word 'or' is always taken in the inclusive sense of $p \vee q$.

2.1.9 Universal operators

Now, any compound proposition that uses any of the operators $\rightarrow$, $\leftrightarrow$, and $\oplus$ can be rewritten as a logically equivalent proposition that uses only $\wedge$, $\vee$, and $\neg$. It is easy to check that $p \rightarrow q$ is logically equivalent to $\neg p \vee q$. (Just make a truth table for $\neg p \vee q$.) Similarly, $p \leftrightarrow q$ can be expressed as $(\neg p \vee q) \wedge (\neg q \vee p)$, So, in a strict logical sense, $\rightarrow$, $\leftrightarrow$, and $\oplus$ are unnecessary. (Nevertheless, they are useful and important, and we won't give them up.)

Even more is true: in a strict logical sense, we could do without the conjunction operator $\wedge$. It is easy to check that $p \wedge q$ is logically equivalent to $\neg(\neg p \vee \neg q)$, so any expression that uses $\wedge$ can be rewritten as one that uses only $\neg$ and $\vee$. Alternatively, we could do without $\vee$ and write everything in terms of $\neg$ and $\wedge$. We shall study some of these rewrite rules in more detail in Section 2.2.

We call a set of operators that can express all operations: ***functionally complete***. More formally we would state the following:

Definition 2.3. A set of logical operators is ***functionally complete*** if and only if all formulas in propositional logic can be rewritten to an equivalent form that uses only operators from the set.

Consider for instance the set $\{\neg, \vee\}$. As shown above the $\wedge$, $\rightarrow$ and $\leftrightarrow$-operators can be expressed using only these operators. In fact all possible operations can be expressed using only $\{\neg, \vee\}$. To prove this you will show in one of the exercises that all possible formulas in propositional logic can be expressed using $\{\neg, \vee, \wedge, \rightarrow, \leftrightarrow\}$. So by showing that we do not need $\wedge$, $\rightarrow$, and $\leftrightarrow$ we can prove that $\{\neg, \vee\}$ is also functionally complete.

2.1.10 Classifying propositions

Certain types of proposition will play a special role in our further work with logic. In particular, we define tautologies, contradictions, and contingencies as follows:

Definition 2.4. A compound proposition is said to be a ***tautology*** if and only if it is *true* for all possible combinations of truth values of the propositional variables which it contains. A compound proposition is said to be a ***contradiction*** if and only if it is *false* for all possible combinations of truth values of the propositional variables which it contains. A compound proposition is said to be a ***contingency*** if and only if it is neither a tautology nor a contradiction.

For example, the proposition $((p \vee q) \wedge \neg q) \rightarrow p$ is a tautology. This can be checked with a truth table:

p	q	$p \vee q$	$\neg q$	$(p \vee q) \wedge \neg q$	$((p \vee q) \wedge \neg q) \rightarrow p$
0	0	0	1	0	1
0	1	1	0	0	1
1	0	1	1	1	1
1	1	1	0	0	1

The fact that all entries in the last column are true tells us that this expression is a tautology. Note that for any compound proposition P, P is a tautology if and only if $\neg P$ is a contradiction. (Here and moving forward, we use uppercase letters to represent compound propositions. P stands for any formula made up of simple propositions, propositional variables, and logical operators.)

Logical equivalence can be defined in terms of tautology:

Definition 2.5. Two compound propositions, P and Q, are said to be ***logically equivalent*** if and only if the proposition $P \leftrightarrow Q$ is a tautology.

The assertion that P is logically equivalent to Q will be expressed symbolically as '$P \equiv Q$'. For example, $(p \rightarrow q) \equiv (\neg p \vee q)$, and $p \oplus q \equiv (p \vee q) \wedge \neg(p \wedge q)$.



What if $P \to Q$ and P is false? From a false premise we can derive any conclusion (check the truth table of $\to$). So if k stands for "I'm the King of the Netherlands", then $k \to Q$ is true for any compound proposition Q. You can substitute anything for Q, and the implication $k \to Q$ will hold. For example, it a logically valid deduction that: If I'm the King of the Netherlands, then unicorns exist. Taking this further, from a contradiction we can derive any conclusion. This is called the ***Principle of Explosion***. (No unicorns were harmed by explaining this principle.)

Exercises



Recall that solutions to some of the exercises start on page 183. Exercises that have a solution are marked with a dagger (†) symbol. We suggest you attempt the exercise first before looking at the solution!

†**1.** Give the three truth tables that define the logical operators $\wedge$, $\vee$, and $\neg$.

†**2.** Some of the following compound propositions are tautologies, some are contradictions, and some are neither (i.e., so are contingencies). In each case, use a truth table to decide to which of these categories the proposition belongs:

a) $(p \wedge (p \to q)) \to q$ **b)** $((p \to q) \wedge (q \to r)) \to (p \to r)$
c) $p \wedge \neg p$ **d)** $(p \vee q) \to (p \wedge q)$
e) $p \vee \neg p$ **f)** $(p \wedge q) \to (p \vee q)$

†**3.** Use truth tables to show that each of the following propositions is logically equivalent to $p \leftrightarrow q$.

a) $(p \to q) \wedge (q \to p)$ **b)** $\neg p \leftrightarrow \neg q$
c) $(p \to q) \wedge (\neg p \to \neg q)$ **d)** $\neg(p \oplus q)$

†**4.** Is $\to$ an associative operation? This is, is $(p \to q) \to r$ logically equivalent to $p \to (q \to r)$?

†**5.** Let p represent the proposition "You leave" and let q represent the proposition "I leave". Express the following sentences as compound propositions using p and q, and show that they are logically equivalent:

a) Either you leave or I do. (Or both!)
b) If you don't leave, I will.

†**6.** Suppose that m represents the proposition "The Earth moves", c represents "The Earth is the centre of the universe", and g represents "Galileo was falsely accused". Translate each of the following compound propositions into English:

a) $\neg g \wedge c$ **b)** $m \to \neg c$
c) $m \leftrightarrow \neg c$ **d)** $(m \to g) \wedge (c \to \neg g)$

†**7.** Give the converse and the contrapositive of each of the following English sentences:

a) If you are good, Sinterklaas brings you toys.
b) If the package weighs more than one kilo, then you need extra postage.

c) If I have a choice, I don't eat courgette.

†**8.** In an ordinary deck of fifty-two playing cards, for how many cards is it true

a) that "This card is a ten and this card is a heart"?

b) that "This card is a ten or this card is a heart"?

c) that "If this card is a ten, then this card is a heart"?

d) that "This card is a ten if and only if this card is a heart"?

†**9.** Define a logical operator $\downarrow$ so that $p \downarrow q$ is logically equivalent to $\neg(p \vee q)$. (This operator is usually referred to as 'NOR', short for 'not or'.) Show that each of the propositions $\neg p$, $p \wedge q$, $p \vee q$, $p \rightarrow q$, $p \leftrightarrow q$, and $p \oplus q$ can be rewritten as a logically equivalent proposition that uses $\downarrow$ as its only operator.

†**10.** For our proof that $\{\neg, \vee\}$ is functionally complete, we need to show that all formulas in propositional logic can be expressed in an equivalent form using only $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$.

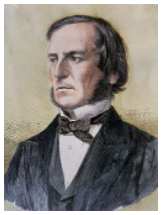
a) How many unique truth tables exist for formulas containing two atoms?

b) Create a function for each of the possible truth tables that uses only the 5 operators listed above.

c) Give an (informal) argument why this means all formulas in propositional logic can be expressed using only these five operators.

2.2 Boolean Algebra

So far we have discussed how to write and interpret propositions. This section deals with *manipulating* them. For this, we need algebra. Ordinary algebra, of the sort taught in high school, is about manipulating numbers, variables that represent numbers, and operators such as $+$ and $\times$ that apply to numbers. Now, we need an algebra that applies to logical values, propositional variables, and logical operators. The first person to think of logic in terms of algebra was the mathematician, George Boole, who introduced the idea in a book that he published in 1854. The algebra of logic is now called ***Boolean algebra*** in his honour.



George Boole (1815–1864) was a largely self-taught British mathematician, philosopher and logician, most of whose short career was spent as the first professor of mathematics at Queen's College, Cork in Ireland. He worked in the fields of differential equations and algebraic logic, and is best known as the author of The Laws of Thought (1854). Among TU Delft students he is best known for the room named after him in the EEMCS building 36.

Boolean logic is credited with laying the foundations for the information age: essentially, computer science. Boole maintained that: "No general method for the solution of questions in the theory of probabilities can be established which does not explicitly recognise, not only the special numerical bases of the science, but also those universal laws of thought which are the basis of all reasoning, and which, whatever they may be as to their essence, are at least mathematical as to their form."

Source: `en.wikipedia.org/wiki/George_Boole`.

The algebra of numbers includes a large number of rules for manipulating expressions. The distributive law, for example, says that $x(y+z) = xy + xz$, where x, y, and z are variables that stand for any numbers or numerical expressions. This law means that whenever you see something of the form $xy + xz$ in a numerical expression, you can substitute $x(y+z)$ without changing the value of the expression, and *vice versa*. Note that the equals sign in $x(y+z) = xy + xz$ means "has the same value as, no matter what numerical values x, y, and z have".

In Boolean algebra, we work with logical values instead of numerical values. There are only two logical values, true and false. We will write these values as $\mathbb{T}$ and $\mathbb{F}$ or 1 and 0. The symbols $\mathbb{T}$ and $\mathbb{F}$ play a similar role in Boolean algebra to the role that constant numbers such as 1 and 3.14159 play in ordinary algebra. Instead of the equals sign, Boolean algebra uses logical equivalence, $\equiv$, which has essentially the same meaning.[5] For example, for propositions p, q, and r, the $\equiv$ operator in $p \wedge (q \wedge r) \equiv (p \wedge q) \wedge r$ means

[5] In ordinary algebra, it is easy to be confused by the equals sign, because it has two very different roles. In an identity such as the distributive law, it means 'is always equal to'. On the other hand, an equation such as $x^2 + 3x = 4$ is a statement that might or might not be true, depending on the value of x. Boolean algebra has two operators, $\equiv$ and $\leftrightarrow$, that play roles similar to the two roles of the equals sign. $\equiv$ is used for identity, whereas $\leftrightarrow$ is used in equations that may or may not be true.

Double negation	$\neg(\neg p) \equiv p$
Excluded middle	$p \vee \neg p \equiv \mathbb{T}$
Contradiction	$p \wedge \neg p \equiv \mathbb{F}$
Identity laws	$\mathbb{T} \wedge p \equiv p$
	$\mathbb{F} \vee p \equiv p$
Idempotent laws	$p \wedge p \equiv p$
	$p \vee p \equiv p$
Commutative laws	$p \wedge q \equiv q \wedge p$
	$p \vee q \equiv q \vee p$
Associative laws	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
	$(p \vee q) \vee r \equiv p \vee (q \vee r)$
Distributive laws	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
DeMorgan's laws	$\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$
	$\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$

Figure 2.2: *Laws of Boolean Algebra. These laws hold for any propositions p, q, and r.*

"has the same value as, no matter what logical values p, q, and r have".

2.2.1 Basics of Boolean Algebra

Many of the rules of Boolean algebra are fairly obvious, if you think a bit about what they mean. Even those that are not obvious can be verified easily by using a truth table. Figure 2.2 lists the most important of these laws. You will notice that all these laws, except the first, come in pairs: each law in the pair can be obtained from the other by interchanging $\wedge$ with $\vee$ and $\mathbb{T}$ with $\mathbb{F}$. This cuts down on the number of facts you have to remember.[6]

Just as an example, let's verify the first rule in the table, the Law of Double Negation. This law is just the old, basic grammar rule that two neg-

[6]It is also an example of a more general fact known as ***duality***, which asserts that given any tautology that uses only the operators $\wedge$, $\vee$, and $\neg$, another tautology can be obtained from it by interchanging $\wedge$ with $\vee$ and $\mathbb{T}$ with $\mathbb{F}$. We won't attempt to prove this here, but we encourage you to try it!

atives make a positive. Although the way this rule applies to English is questionable, if you look at how it is used—no matter what the grammarian says, "I can't get no satisfaction" doesn't really mean "I can get satisfaction"—the validity of the rule in logic can be verified just by computing the two possible cases: when p is true and when p is false. When p is true, then by the definition of the $\neg$ operator, $\neg p$ is false. But then, again by the definition of $\neg$, the value of $\neg(\neg p)$ is true, which is the same as the value of p. Similarly, in the case where p is false, $\neg(\neg p)$ is also false. Organized into a truth table, this argument takes the rather simple form

p	$\neg p$	$\neg(\neg p)$
0	1	0
1	0	1

The fact that the first and last columns are identical shows the logical equivalence of p and $\neg(\neg p)$. The point here is not just that $\neg(\neg p) \equiv p$, but also that this logical equivalence is valid because it can be verified computationally based just on the relevant definitions. Its validity does *not* follow from the fact that "it's obvious" or "it's a well-known rule of grammar".



Students often ask "Why do I have to prove something when it's obvious?" The point is that logic—and mathematics more generally—is its own little world with its own set of rules. Although this world is related somehow to the real world, when you say that something is obvious (in the real world), you aren't playing by the rules of the world of logic. The real *magic* of mathematics is that by playing by its rules, you can come up with things that are decidedly not obvious, but that still say something about the real world or the computational world—often, something interesting and important.

Each of the rules in Figure 2.2 can be verified in the same way, by making a truth table to check all the possible cases. In one of the pencasts of this course we further discuss how to check the equivalence of two propositions using truth tables.

2.2.2 Substitution laws

It's important to understand that the propositional variables in the laws of Boolean algebra can stand for any propositions, including compound propositions. It is not just true, as the Double Negation Law states, that

$\neg(\neg p) \equiv p$. It is also true that $\neg(\neg q) \equiv q$, that $\neg(\neg(p \wedge q)) \equiv (p \wedge q)$, that $\neg(\neg(p \rightarrow (q \wedge \neg p))) \equiv (p \rightarrow (q \wedge \neg p))$, and an infinite number of other statements of the same form. Here, a 'statement of the same form' is one that can be obtained by substituting something for p in both places where it occurs in $\neg(\neg p) \equiv p$. How can I be sure that all these infinitely many statements are valid when all that I've checked is one little two-line truth table? The answer is that any given proposition, Q, no matter how complicated, has a particular truth value, either true or false. So, the question of the validity of $\neg(\neg Q) \equiv Q$ always reduces to one of the two cases I already checked in the truth table. (Note that for this argument to be valid, the same Q must be substituted for p in every position where it occurs.) While this argument may be 'obvious', it is not exactly a proof, but for now we will just accept the validity of the following theorem:

Theorem 2.1 (First Substitution Law)**.** *Suppose that Q is any proposition, and that p is a propositional variable. Consider any tautology. If (Q) is substituted for p in all places where p occurs in the tautology, then the result is also a tautology.*

Since logical equivalence is defined in terms of tautology, it is also true that when (Q) is substituted for p in a logical equivalence, the result is again a logical equivalence.[7]

The First Substitution Law lets you do algebra! For example, you can substitute $p \rightarrow q$ for p in the law of double negation, $\neg(\neg p) \equiv p$. This allows you to 'simplify' the expression $\neg(\neg(r \rightarrow q))$ to $r \rightarrow q$ with confidence that the resulting expression has the same logical value as the expression you started with. (That's what it means for $\neg(\neg(r \rightarrow q))$ and $r \rightarrow q$ to be logically equivalent.) You can play similar tricks with all the laws in Figure 2.2. Even more important is the Second Substitution Law, which says that you can substitute an expression for a logically equivalent expression, wherever it occurs. Once again, we will accept this as a theorem without trying to prove it here. It is surprisingly hard to put this law into words:

Theorem 2.2 (Second Substitution Law)**.** *Suppose that P and Q are any propositions such that $P \equiv Q$. Suppose that R is any compound proposition in which (P) occurs as a sub-proposition. Let R' be the proposition that is obtained by substituting (Q) for that occurrence of (P) in R. Then $R \equiv R'$.*

Note that in this case, the theorem does not require (Q) to be substituted for *every* occurrence of (P) in R. You are free to substitute for one, two,

[7] I've added parentheses around Q here for technical reasons. Sometimes, the parentheses are necessary to make sure that Q is evaluated as a whole, so that its final value is used in place of p. As an example of what can go wrong, consider $q \wedge r$. If this is substituted literally for p in $\neg(\neg p)$, without parentheses, the result is $\neg(\neg q \wedge r)$. But this expression means $\neg((\neg q) \wedge r)$, which is *not* equivalent to $q \wedge r$. Did we say to always write parentheses if you're in doubt? See page 9.

or as many occurrences of (P) as you like, and the result is still logically equivalent to R.

The Second Substitution Law allows us to use the logical equivalence $\neg(\neg p) \equiv p$ to 'simplify' the expression $q \rightarrow (\neg(\neg p))$ by substituting $\neg(\neg p)$ for p. The resulting expression, $q \rightarrow p$, is logically equivalent to the original $q \rightarrow (\neg(\neg p))$. Once again, we have to be careful about parentheses: The fact that $p \vee p \equiv p$ does *not* allow us to rewrite $q \wedge p \vee p \wedge r$ as $q \wedge p \wedge r$. The problem is that $q \wedge p \vee p \wedge r$ means $(q \wedge p) \vee (p \wedge r)$, so that $(p \vee p)$ is not a sub-expression. This again underlines the importance of always writing parentheses in your propositional formulas.

2.2.3 Simplifications

The final piece of algebra in Boolean algebra is the observation that we can chain logical equivalences together. That is, from $P \equiv Q$ and $Q \equiv R$, it follows that $P \equiv R$. This is really just a consequence of the Second Substitution Law. The equivalence $Q \equiv R$ allows us to substitute R for Q in the statement $P \equiv Q$, giving $P \equiv R$. (Remember that, by Definition 2.5, logical equivalence is defined in terms of a proposition.) This means that we can show that two compound propositions are logically equivalent by finding a chain of logical equivalences that lead from one to the other.

Here is an example of such a chain of logical equivalences:

$$
\begin{aligned}
p \wedge (p \rightarrow q) &\equiv p \wedge (\neg p \vee q) && \text{definition of } p \rightarrow q, \text{ Theorem 2.2} \\
&\equiv (p \wedge \neg p) \vee (p \wedge q) && \text{Distributive Law} \\
&\equiv \mathbb{F} \vee (p \wedge q) && \text{Law of Contradiction, Theorem 2.2} \\
&\equiv (p \wedge q) && \text{Identity Law}
\end{aligned}
$$

Each step in the chain has its own justification. In several cases, a substitution law is used without stating as much. In the first line, for example, the definition of $p \rightarrow q$ is that $p \rightarrow q \equiv \neg p \vee q$. The Second Substitution Law allows us to substitute $(\neg p \vee q)$ for $(p \rightarrow q)$. In the last line, we implicitly applied the First Substitution Law to the Identity Law, $\mathbb{F} \vee p \equiv p$, to obtain $\mathbb{F} \vee (p \wedge q) \equiv (p \wedge q)$.

The chain of equivalences in the above example allows us to conclude that $p \wedge (p \rightarrow q)$ is logically equivalent to $p \wedge q$. This means that if you were to make a truth table for these two expressions, the truth values in the column for $p \wedge (p \rightarrow q)$ would be identical to those in the column for $p \wedge q$. We know this without actually making the table. Don't believe it? Go ahead and make the truth table. In this case, the table is only be four lines long and easy enough to make. But Boolean algebra can be applied in cases

where the number of propositional variables is too large for a truth table to be practical.



Let's do another example. Recall that a compound proposition is a tautology if it is true for all possible combinations of truth values of the propositional variables that it contains. But another way of saying the same thing is that P is a tautology if $P \equiv \mathbb{T}$. So, we can prove that a compound proposition, P, is a tautology by finding a chain of logical equivalences leading from P to $\mathbb{T}$. For example:

$$
\begin{array}{lll}
((p \vee q) \wedge \neg p) \rightarrow q & & \\
\quad \equiv (\neg((p \vee q) \wedge \neg p)) \vee q & & \text{definition of } \rightarrow \\
\quad \equiv (\neg(p \vee q) \vee \neg(\neg p)) \vee q & & \text{DeMorgan's Law, Theorem 2.2} \\
\quad \equiv (\neg(p \vee q) \vee p) \vee q & & \text{Double Negation, Theorem 2.2} \\
\quad \equiv (\neg(p \vee q)) \vee (p \vee q) & & \text{Associative Law for } \vee \\
\quad \equiv \mathbb{T} & & \text{Law of Excluded Middle}
\end{array}
$$

From this chain of equivalences, we can conclude that $((p \vee q) \wedge \neg p) \rightarrow q$ is a tautology.

Now, it takes some practice to look at an expression and see which rules can be applied to it; to see $(\neg(p \vee q)) \vee (p \vee q)$ as an application of the law of the excluded middle for example, you need to mentally substitute $(p \vee q)$ for p in the law as it is stated in Figure 2.2. Often, there are several rules that apply, and there are no definite guidelines about which one you should try. This is what makes algebra something of an art.

2.2.4 More rules of Boolean algebra

It is certainly not true that all possible rules of Boolean algebra are given in Figure 2.2. For one thing, there are many rules that are easy consequences of the rules that are listed there. For example, although the table asserts only that $\mathbb{F} \vee p \equiv p$, it is also true that $p \vee \mathbb{F} \equiv p$. This can be checked directly or by a simple calculation:

$$
\begin{array}{lll}
p \vee \mathbb{F} \equiv \mathbb{F} \vee p & & \text{Commutative Law} \\
\quad \equiv p & & \text{Identity Law as given in the table}
\end{array}
$$

Additional rules can be obtained by applying the Commutative Law to other rules in the table, and we will use such rules freely in the future.

Another sort of easy extension can be applied to the Associative Law, $(p \vee q) \vee r \equiv p \vee (q \vee r)$. The law is stated for the $\vee$ operator applied to three terms, but it generalizes to four or more terms. For example

$$
\begin{aligned}
&((p \vee q) \vee r) \vee s \\
&\qquad \equiv (p \vee q) \vee (r \vee s) \qquad \text{by the Associative Law for three terms} \\
&\qquad \equiv p \vee (q \vee (r \vee s)) \qquad \text{by the Associative Law for three terms}
\end{aligned}
$$

We will, of course, often write this expression as $p \vee q \vee r \vee s$, with no parentheses at all, knowing that wherever we put the parentheses the value is the same.



One other thing that you should keep in mind is that rules can be applied in either direction. The Distributive Law, for example, allows you to distribute the p in $p \vee (q \wedge \neg p)$ to get $(p \vee q) \wedge (p \vee \neg p)$. But it can also be used in reverse to 'factor out' a term, as when you start with $(q \vee (p \rightarrow q)) \wedge (q \vee (q \rightarrow p))$ and factor out the q to get $q \vee ((p \rightarrow q) \wedge (q \rightarrow p))$.

So far in this section, we have been working with the laws of Boolean algebra without saying much about what they mean or why they are reasonable. Of course, you can apply the laws in calculations without understanding them. But if you want to figure out *which* calculations to do, you need some understanding. Most of the laws are clear enough with a little thought. For example, if we already know that q is false, then $p \vee q$ will be true when p is true and false when p is false. That is, $p \vee \mathbb{F}$ has the same logical value as p. But that's just what the Identity Law for $\vee$ says. A few of the laws need more discussion.

The Law of the Excluded Middle, $p \vee \neg p \equiv \mathbb{T}$, says that given any proposition p, at least one of p or $\neg p$ must be true. Since $\neg p$ is true exactly when p is false, this is the same as saying that p must be either true or false. There is no middle ground. The Law of Contradiction, $p \wedge \neg p \equiv \mathbb{F}$, says that it is not possible for *both* p and $\neg p$ to be true. Both of these rules are obvious.



There are some who set out to question the law of there being no middle ground. Already in the 1920's people like Tarski (who we will meet later) talked about other forms of logic where another value representing 'unknown' or 'not proven' also exists. You can also see this in some programming languages where they are referred to as 'tri-state booleans'.



These so-called non-standard logics have been developed and have also lead to things like 'fuzzy logic', which some consider quite controversial. Lotfi Zadeh is credited as the first person to refer to this type of logic as fuzzy logic in his work on 'fuzzy sets' in 1965. Zadeh was later quoted as saying: "Not being afraid to get embroiled in controversy. ... That's part of my character, too. I can be very stubborn. That's probably been beneficial for the development of Fuzzy Logic."

Source: `en.wikipedia.org/wiki/Lotfi_A._Zadeh`

The Distributive Laws cannot be called obvious, but a few examples can show that they are reasonable. Consider the statement, "This card is the ace of spades or clubs." Clearly, this is equivalent to "This card is the ace of spaces or this card is the ace of clubs." But this is just an example of the first distributive law! For, let a represent the proposition "This card is an ace", let s represent "This card is a spade" and let c represent "This card is a club". Then "This card is the ace of spades or clubs" can be translated into logic as $a \wedge (s \vee c)$, while "This card is the ace of spades or this card is the ace of clubs" becomes $(a \wedge s) \vee (a \wedge c)$. And the distributive law assures us that $a \wedge (s \vee c) \equiv (a \wedge s) \vee (a \wedge c)$. The second distributive law tells us, for example, that "This card is either a joker or is the ten of diamonds" is logically equivalent to "This card is either a joker or a ten, and it is either a joker or a diamond". That is, $j \vee (t \wedge d) \equiv (j \vee t) \wedge (j \vee d)$. The distributive laws are powerful tools and you should keep them in mind whenever you are faced with a mixture of $\wedge$ and $\vee$ operators.

DeMorgan's Laws must also be less than obvious, since people often get them wrong. Fortunately you get to practice them both in *Reasoning & Logic*, as well as in *Computer Organisation*, so you will soon get them right. More importantly perhaps they do also make sense. When considering $\neg(p \wedge q)$, you should ask yourself, how can 'p and q' *fail* to be true. It will fail to be

true if either p is false *or* if q is false (or both). That is, $\neg(p \wedge q)$ is equivalent to $(\neg p) \vee (\neg q)$. Consider the sentence "A raven is large and black." If a bird is *not* large and black, then it is not a raven. But what exactly does it mean to be '*not* (*large and black*)'? How can you tell whether the assertion 'not (large and black)' is true of something? This will be true if it is either not large *or* not black. (It doesn't have to be both—it could be large and white, it could be small and black.) Similarly, for 'p or q' to fail to be true, *both* p and q must be false. That is, $\neg(p \vee q)$ is equivalent to $(\neg p) \wedge (\neg q)$. This is DeMorgan's second law.

Recalling that $p \rightarrow q$ is equivalent to $(\neg p) \vee q$, we can apply DeMorgan's law to obtain a formula for the negation an implication:

$$\begin{aligned} \neg(p \rightarrow q) &\equiv \neg((\neg p) \vee q) \\ &\equiv (\neg(\neg p)) \wedge (\neg q) \\ &\equiv p \wedge \neg q \end{aligned}$$

That is, $p \rightarrow q$ is false exactly when both p is true and q is false. For example, the negation of "If you have an ace, you win" is "You have an ace, and you don't win". Think of it this way: if you had an ace and you didn't win, then the statement "If you have an ace, you win" was not true.

Exercises

1. Construct truth tables to demonstrate the validity of each of the distributive laws.

2. Construct the following truth tables:

a) Construct truth tables to demonstrate that $\neg(p \wedge q)$ is **not** logically equivalent to $(\neg p) \wedge (\neg q)$.

b) Construct truth tables to demonstrate that $\neg(p \vee q)$ is **not** logically equivalent to $(\neg p) \vee (\neg q)$.

c) Construct truth tables to demonstrate the validity of both DeMorgan's Laws.

3. Construct truth tables to demonstrate that $\neg(p \rightarrow q)$ is **not** logically equivalent to any of the following.

a) $(\neg p) \rightarrow (\neg q)$

b) $(\neg p) \rightarrow q$

c) $p \rightarrow (\neg q)$

Refer back to this section for a formula that **is** logically equivalent to $\neg(p \rightarrow q)$.

†**4.** Is $\neg(p \leftrightarrow q)$ logically equivalent to $(\neg p) \leftrightarrow (\neg q)$?

5. In the algebra of numbers, there is a distributive law of multiplication over addition: $x(y + z) = xy + xz$. What would a distributive law of addition over multiplication look like? Is it a valid law in the algebra of numbers?

6. The distributive laws given in Figure 2.2 are sometimes called the *left* distributive laws. The ***right distributive laws*** say that $(p \vee q) \wedge r \equiv (p \wedge r) \vee (q \wedge r)$ and that $(p \wedge q) \vee r \equiv (p \vee r) \wedge (q \vee r)$. Show that the right distributive laws are also valid laws of Boolean algebra. (Note: In practice, both the left and the right distributive laws are referred to simply as the distributive laws, and both can be used freely in proofs.)

7. Show that $p \wedge (q \vee r \vee s) \equiv (p \wedge q) \vee (p \wedge r) \vee (p \wedge s)$ for any propositions p, q, r, and s. In words, we can say that conjunction distributes over a disjunction of three terms. (Recall that the $\wedge$ operator is called conjunction and $\vee$ is called disjunction.) Translate into logic and verify the fact that conjunction distributes over a disjunction of four terms. Argue that, in fact, conjunction distributes over a disjunction of any number of terms.

8. There are two additional basic laws of logic, involving the two expression $p \wedge \mathbb{F}$ and $p \vee \mathbb{T}$. What are the missing laws? Show that your answers are, in fact, laws.

9. For each of the following pairs of propositions, show that the two propositions are logically equivalent by finding a chain of equivalences from one to the other. State which definition or law of logic justifies each equivalence in the chain.

a) $p \wedge (q \wedge p)$, $p \wedge q$ **b)** $(\neg p) \rightarrow q$, $p \vee q$
c) $(p \vee q) \wedge \neg q$, $p \wedge \neg q$ **d)** $p \rightarrow (q \rightarrow r)$, $(p \wedge q) \rightarrow r$
e) $(p \rightarrow r) \wedge (q \rightarrow r)$, $(p \vee q) \rightarrow r$ **f)** $p \rightarrow (p \wedge q)$, $p \rightarrow q$

†**10.** For each of the following compound propositions, find a simpler proposition that is logically equivalent. Try to find a proposition that is as simple as possible.

a) $(p \wedge q) \vee \neg q$ **b)** $\neg(p \vee q) \wedge p$ **c)** $p \rightarrow \neg p$
d) $\neg p \wedge (p \vee q)$ **e)** $(q \wedge p) \rightarrow q$ **f)** $(p \rightarrow q) \wedge (\neg p \rightarrow q)$

†**11.** Express the negation of each of the following sentences in natural English:

a) It is sunny and cold.
b) I will have stroopwafel or I will have appeltaart.
c) If today is Tuesday, this is Belgium.
d) If you pass the final exam, you pass the course.

12. Apply one of the laws of logic to each of the following sentences, and rewrite it as an equivalent sentence. State which law you are applying.

a) I will have coffee and stroopwafel or appeltaart.
b) He has neither talent nor ambition.
c) You can have oliebollen, or you can have oliebollen.

13. Suppose it is simultaneously true that "All lemons are yellow" and "Not all lemons are yellow". Derive the conclusion "Unicorns exist". (If you get stuck, check out `en.wikipedia.org/wiki/Principle_of_explosion`.)

2.3 Application: Logic Circuits

As we saw in Chapter 1, computers have a reputation—not always deserved—for being 'logical'. But fundamentally, deep down, they are made of logic in a very real sense. The building blocks of computers are ***logic gates***, which

are electronic components that compute the values of simple propositions such as $p \wedge q$ and $\neg p$. (Each gate is in turn built of even smaller electronic components called transistors, but this needn't concern us here: see the course *Computer Organisation*.)



Don't worry, logic circuits will be examined in *Computer Organisation*, not in *Reasoning & Logic*. They are a good example and application of propositional logic, and that's why we're talking about them in this section. Normal forms (Section 2.3.4) are definitely on the syllabus, however, so pay attention!

2.3.1 Logic gates

A wire in a computer can be in one of two states, which we can think of as being *on* and *off*. These two states can be naturally associated with the Boolean values $\mathbb{T}$ and $\mathbb{F}$. When a computer computes, the multitude of wires inside it are turned on and off in patterns that are determined by certain rules. The rules involved can be most naturally expressed in terms of logic. A simple rule might be: "turn wire C on whenever wire A is on and wire B is on". This rule can be implemented in hardware as an ***AND gate***. An AND gate is an electronic component with two input wires and one output wire, whose job is to turn its output on when both of its inputs are on and to turn its output off in any other case. If we associate 'on' with $\mathbb{T}$ and 'off' with $\mathbb{F}$, and if we give the names A and B to the inputs of the gate, then the gate computes the value of the logical expression $A \wedge B$. In effect, A is a proposition with the meaning "the first input is on", and B is a proposition with the meaning "the second input is on". The AND gate functions to ensure that the output is described by the proposition $A \wedge B$. That is, the output is on if and only if the first input is on and the second input is on.

As you hopefully know from *Computer Organisation*, an ***OR gate*** is an electronic component with two inputs and one output which turns its output on if either (or both) of its inputs is on. If the inputs are given names A and B, then the OR gate computes the logical value of $A \vee B$. A ***NOT gate*** has one input and one output, and it turns its output off when the input is on and on when the input is off. If the input is named A, then the NOT gate computes the value of $\neg A$.



As we mentioned earlier, other textbooks might use different notations to represent a negation. For instance a bar over the variable $\bar{x}$ or a $\sim$ symbol. In digital logic (and thus in your *Computer Organisation* course) you will also often find the $+$ symbol to represent an 'or' and a $\cdot$ (dot) symbol to represent an 'and'.

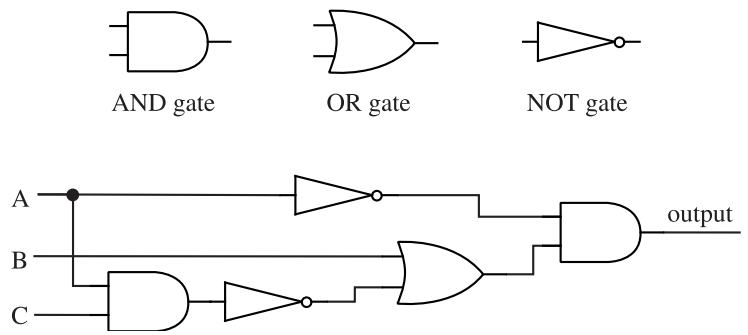


Figure 2.3: *The standard symbols for the three basic logic gates, and a logic circuit that computes the value of the logical expression* $(\neg A) \wedge (B \vee \neg(A \wedge C))$. *The input wires to each logic gate are on the left, with the output wire on the right. Note that when wires cross each other in a diagram such as this, the wires don't actually intersect unless there is a black circle at the point where they cross.*

Other types of logic gates are, of course, possible. Gates could be made to compute $A \rightarrow B$ or $A \oplus B$, for example. However, any computation that can be performed by logic gates can be done using only AND, OR, and NOT gates, as we will see below. (In practice, however, NAND gates and NOR gates, which compute the values of $\neg(A \wedge B)$ and $\neg(A \vee B)$ respectively, are often used because they are easier to build from transistors than AND and OR gates.)

2.3.2 Combining gates to create circuits

The three types of logic gates are represented by standard symbols, as shown in Figure 2.3. Since the inputs and outputs of logic gates are just wires carrying on/off signals, logic gates can be wired together by connecting outputs from some gates to inputs of other gates. The result is a ***logic circuit***. An example is also shown in Figure 2.3.

The logic circuit in the figure has three inputs, labeled A, B, and C. The circuit computes the value of the compound proposition $(\neg A) \wedge (B \vee \neg(A \wedge C))$. That is, when A represents the proposition "the input wire labeled A is on," and similarly for B and C, then the output of the circuit is on if and only if the value of the compound proposition $(\neg A) \wedge (B \vee \neg(A \wedge C))$ is true.

Given any compound proposition made from the operators $\wedge$, $\vee$, and $\neg$, it is possible to build a logic circuit that computes the value of that propos-

ition. The proposition itself is a blueprint for the circuit. As noted in Section 2.1, every logical operator that we have encountered can be expressed in terms of $\wedge$, $\vee$, and $\neg$, so in fact every compound proposition that we know how to write can be computed by a logic circuit.

Given a proposition constructed from $\wedge$, $\vee$, and $\neg$ operators, it is easy to build a circuit to compute it. First, identify the main operator in the proposition—the one whose value will be computed *last*. Consider $(A \vee B) \wedge \neg(A \wedge B)$. This circuit has two input values, A and B, which are represented by wires coming into the circuit. The circuit has an output wire that represents the computed value of the proposition. The main operator in $(A \vee B) \wedge \neg(A \wedge B)$, is the first $\wedge$, which computes the value of the expression as a whole by combining the values of the subexpressions $A \vee B$ and $\neg(A \wedge B)$. This $\wedge$ operator corresponds to an AND gate in the circuit that computes the final output of the circuit.

Once the main operator has been identified and represented as a logic gate, you just have to build circuits to compute the input or inputs to that operator. In the example, the inputs to the main AND gate come from two subcircuits. One subcircuit computes the value of $A \vee B$ and the other computes the value of $\neg(A \wedge B)$. Building each subcircuit is a separate problem, but smaller than the problem you started with. Eventually, you'll come to a gate whose input comes directly from one of the input wires—A or B in this case—instead of from a subcircuit.

2.3.3 From circuits to propositions

So, every compound proposition is computed by a logic circuit with one output wire. Is the reverse true? That is, given a logic circuit with one output, is there a proposition that expresses the value of the output in terms of the values of the inputs? Not quite. When you wire together some logic gates to make a circuit, there is nothing to stop you from introducing feedback loops. A feedback loop occurs when the output from a gate is connected—possibly through one or more intermediate gates—back to an input of the same gate. Figure 2.5 shows an example of a circuit with a feedback loop. Feedback loops cannot be described by compound propositions, basically because there is no place to start, no input to associate with a propositional variable. But feedback loops are the only thing that can go wrong. A logic circuit that does not contain any feedback loops is called a ***combinatorial logic circuit***. Every combinatorial logic circuit with just one output computes the value of some compound proposition. The propositional variables in the compound proposition are just names associated with the input wires of the circuit. (Of course, if the circuit has more than one output, you can simply use a different proposition for each output.)

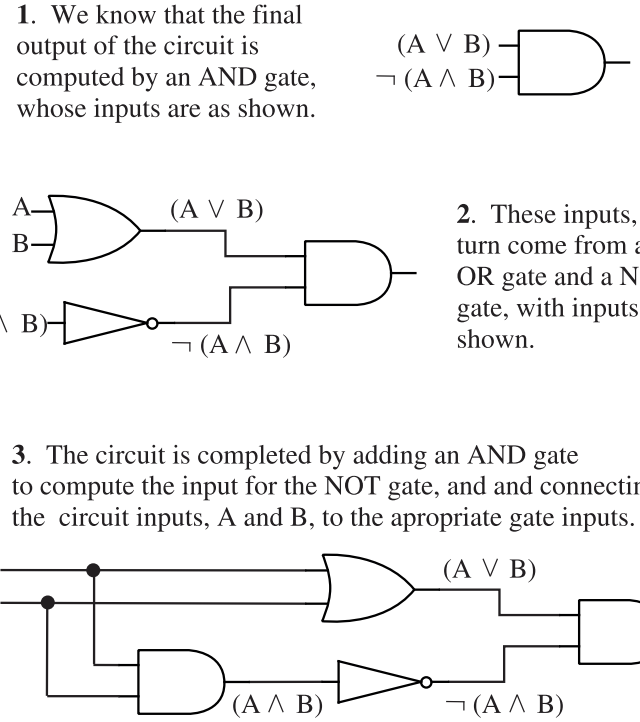


Figure 2.4: *Stages in the construction of a circuit that computes the compound proposition* $(A \vee B) \wedge \neg(A \wedge B)$.

The key to understanding why this is true is to note that each wire in the circuit—not just the final output wire—represents the value of some proposition. Furthermore, once you know which proposition is represented by each input wire to a gate, it's obvious what proposition is represented by the output: You just combine the input propositions with the appropriate $\wedge$, $\vee$, or $\neg$ operator, depending on what type of gate it is. To find the proposition associated with the final output, you just have to start from the inputs and move through the circuit, labeling the output wire of each gate with the proposition that it represents. Figure 2.6 illustrates this process.

2.3.4 Disjunctive Normal Form

Compound propositions, then, correspond naturally with combinatorial logic circuits. But we have still not quite settled the question of just how

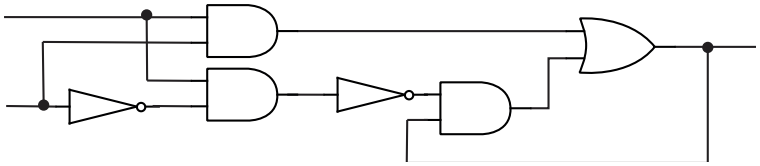


Figure 2.5: *This circuit contains a feedback loop, so it is not a combinatorial logic circuit. The feedback loop includes the* AND *gate and the* OR *gate on the right. This circuit does not compute the value of a compound proposition. This circuit does, however, play an important role in computer memories, since it can be used to store a logical value.*

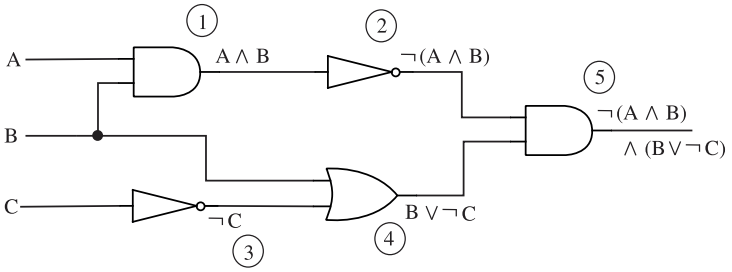


Figure 2.6: *Finding the proposition whose value is computed by a combinatorial logic circuit. Each wire in the circuit is labeled with the proposition that it represents. The numbering of the labels shows one of the orders in which they can be associated with the wires. The circuit as a whole computes the value of* $\neg(A \wedge B) \wedge (B \vee \neg C)$.

powerful these circuits and propositions are. We've looked at a number of logical operators and noted that they can all be expressed in terms of $\wedge$, $\vee$, and $\neg$. But might there be other operators that cannot be so expressed? Equivalently, might there be other types of logic gates—possibly with some large number of inputs—whose computations cannot be duplicated with AND, OR, and NOT gates? Any logical operator or logic gate computes a value for each possible combination of logical values of its inputs. We could always make a truth table showing the output for each possible combination of inputs. As it turns out, given *any* such truth table, it is possible to find a proposition, containing only the $\wedge$, $\vee$, and $\neg$ operators, whose value for each combination of inputs is given precisely by that table.

To see why this is true, it is useful to introduce a particular type of compound proposition. Define a ***simple term*** to be either a propositional vari-

able or the negation of a propositional variable. A conjunction of simple terms would then consist of one or more simple terms put together with $\wedge$ operators. (A 'conjunction of one simple term' is just a single simple term by itself. This might not make grammatical sense, but it's the way mathematicians think.) Some examples of conjunctions of simple terms would be $p \wedge q$, p, $\neg q$, and $p \wedge \neg r \wedge \neg w \wedge s \wedge t$. Finally, we can take one or more such conjunctions and join them into a 'disjunction of conjunctions of simple terms'. This is the type of compound proposition we need. We can avoid some redundancy by assuming that no propositional variable occurs more than once in a single conjunction (since $p \wedge p$ can be replaced by p, and if p and $\neg p$ both occur in a conjunction, then the value of the conjuction is false, and it can be eliminated.) We can also assume that the same conjunction does not occur twice in the disjunction.



Normal forms are part of the syllabus for *Reasoning & Logic*. These normal forms, such as Disjunctive Normal Form (this subsection) and Conjunctive Normal Form (see the exercises), are important in propositional logic. There are normal forms for other logics, too, such as for *predicate logic* which we'll look at in the next Section 2.4.

Definition 2.6. A compound proposition is said to be in ***disjunctive normal form***, or DNF, if it is a disjunction of conjunctions of simple terms, and if, furthermore, each propositional variable occurs at most once in each conjunction and each conjunction occurs at most once in the disjunction.

Using p, q, r, s, A, and B as propositional variables, here are a few examples of propositions that are in disjunctive normal form:

$$(p \wedge q \wedge r) \vee (p \wedge \neg q \wedge r \wedge s) \vee (\neg p \wedge \neg q)$$
$$(p \wedge \neg q)$$
$$(A \wedge \neg B) \vee (\neg A \wedge B)$$
$$p \vee (\neg p \wedge q) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge \neg q \wedge \neg r \wedge w)$$

Propositions in DNF are just what we need to deal with input/output tables of the type that we have been discussing. Any such table can be computed by a proposition in disjunctive normal form. It follows that it is possible to build a circuit to compute that table using only AND, OR, and NOT gates.

Theorem 2.3. *Consider a table that lists a logical output value for every combination of values of several propositional variables. Assume that at least one of the output values is true. Then there is a proposition containing those variables such that the value of the proposition for each possible combination of the values of the*

variables is precisely the value specified in the table. It is possible to choose the proposition to be in disjunctive normal form.

Proof. Consider any row in the table for which the output value is $\mathbb{T}$. Form a conjunction of simple terms as follows: For each variable, p, whose value is $\mathbb{T}$ in that row, include p itself in the conjunction; for each variable, q, whose value is $\mathbb{F}$ in the row, include $\neg q$ in the conjunction. The value of this conjunction is $\mathbb{T}$ for the combination of variable values given in that row of the table, since each of the terms in the conjuction is true for that combination of variables. Furthermore, for any *other* possible combination of variable values, the value of the conjunction will be $\mathbb{F}$, since at least one of the simple terms in the conjunction will be false.

Take the disjunction of all such conjunctions constructed in this way, for each row in the table where the output value is true. This disjunction has the value $\mathbb{T}$ if and only if one of the conjunctions that make it up has the value $\mathbb{T}$—and that is precisely when the output value specified by the table is $\mathbb{T}$. So, this disjunction of conjunctions satisfies the requirements of the theorem. □



This is the first proof of a non-trivial claim that we've seen. You will learn about theorems and proofs, and proof techniques, at the end of this chapter and in Chapter 3.

As an example, consider the table in Figure 2.7. This table specifies a desired output value for each possible combination of values for the propositional variables p, q, and r. Look at the second row of the table, where the output value is true. According to the proof of the theorem, this row corresponds to the conjunction $(\neg p \wedge \neg q \wedge r)$. This conjunction is true when p is false, q is false, and r is true; in all other cases it is false, since in any other case at least one of the terms $\neg p$, $\neg q$, or r is false. The other two rows where the output is true give two more conjunctions. The three conjunctions are combined to produce the DNF proposition $(\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)$. This proposition computes all the output values specified in the table. Using this proposition as a blueprint, we get a logic circuit whose outputs match those given in the table.

Now, given any combinatorial logic circuit, there are many other circuits that have the same input/output behaviour. When two circuits have the same input/output table, the compound propositions associated with the two circuits are logically equivalent. To put this another way, propositions that are logically equivalent produce circuits that have the same input/output behaviour. As a practical matter, we will usually prefer the circuit that

p	q	r	output	
$\mathbb{F}$	$\mathbb{F}$	$\mathbb{F}$	$\mathbb{F}$	
$\mathbb{F}$	$\mathbb{F}$	$\mathbb{T}$	$\mathbb{T}$	$(\neg p \wedge \neg q \wedge r)$
$\mathbb{F}$	$\mathbb{T}$	$\mathbb{F}$	$\mathbb{F}$	
$\mathbb{F}$	$\mathbb{T}$	$\mathbb{T}$	$\mathbb{T}$	$(\neg p \wedge q \wedge r)$
$\mathbb{T}$	$\mathbb{F}$	$\mathbb{F}$	$\mathbb{F}$	
$\mathbb{T}$	$\mathbb{F}$	$\mathbb{T}$	$\mathbb{F}$	
$\mathbb{T}$	$\mathbb{T}$	$\mathbb{F}$	$\mathbb{F}$	
$\mathbb{T}$	$\mathbb{T}$	$\mathbb{T}$	$\mathbb{T}$	$p \wedge q \wedge r$

Figure 2.7: *An input/output table specifying a desired output for each combination of values of the propositional variables p, q, and r. Each row where the output is $\mathbb{T}$ corresponds to a conjunction, shown next to that row in the table. The disjunction of these conjunctions is a proposition whose output values are precisely those specified by the table.*

is simpler. The correspondence between circuits and propositions allows us to apply Boolean algebra to the simplification of circuits.



Our preference for simpler applies to compound propositions, whether or not they correspond to circuits. We usually prefer the equivalent form of the proposition that is simpler. Any proposition has an equivalent proposition in DNF. So when proving a theorem about compound propositions, it is sufficient to consider only DNF propositions. This can make the proof easier to write.

For example, consider the DNF proposition corresponding to the table in Figure 2.7. In $(\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)$, we can factor $(q \wedge r)$ from the last two terms, giving $(\neg p \wedge \neg q \wedge r) \vee ((\neg p \vee p) \wedge (q \wedge r))$. Since $\neg p \vee p \equiv \mathbb{T}$, and $\mathbb{T} \wedge Q \equiv Q$ for any proposition Q, this can be simplified to $(\neg p \wedge \neg q \wedge r) \vee (q \wedge r)$. Again, we can apply the distributive law to this to factor out an r, giving $((\neg p \wedge \neg q) \vee q) \wedge r)$. This compound proposition is logically equivalent to the one we started with, but implementing it in a circuit requires only five logic gates, instead of the ten required by the original proposition.[8]

[8]No, we didn't count wrong. There are eleven logical operators in the original expression,

If you start with a circuit instead of a proposition, it is often possible to find the associated proposition, simplify it using Boolean algebra, and use the simplified proposition to build an equivalent circuit that is simpler than the original. And simplifying a proposition to DNF is often a sensible approach.



One way to simplify propositions is using a ***Karnaugh-map*** (or *K-map* for short) as you will learn in *Computer Organisation*. Using a K-map you can find what they will call a 'minimal sum of products'. Notice that a sum of products is just a proposition written in DNF. For the course of *Reasoning & Logic* we may ask you to translate propositions to a DNF form. You can then choose to either do so using rewrite rules, but you are also free to use a K-map if you prefer. In one of the pencasts of this course we show how both methods lead to a result in DNF: `youtu.be/GwVngCU9eYY`.

2.3.5 Binary addition

All this explains nicely the relationship between logic and circuits, but it doesn't explain why logic circuits should be used in computers in the first place. Part of the explanation is found in the fact that computers use binary numbers. A binary number is a string of zeros and ones. Binary numbers are easy to represent in an electronic device like a computer: Each position in the number corresponds to a wire. When the wire is on, it represents one; when the wire is off, it represents zero. When we are thinking in terms of logic, the same states of the wire represent true and false, but either representation is just an interpretation of the reality, which is a wire that is on or off. The question is whether the interpretation is fruitful.

Once wires are thought of as representing zeros and ones, we can build circuits to do computations with binary numbers. Which computations? Any that we want! If we know what the answer should be for each combination of inputs, then by Theorem 2.3 we can build a circuit to compute that answer. Of course, the procedure described in that theorem is only practical for small circuits, but small circuits can be used as building blocks to make all the calculating circuits in a computer.

For example, let's look at binary addition. To add two ordinary, decimal

but you can get by with ten gates in the circuit: Use a single NOT gate to compute $\neg p$, and connect the output of that gate to two different AND gates. Reusing the output of a logic gate is an obvious way to simplify circuits that does not correspond to any operation on propositions.

A	B	C	output
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

A	B	C	output
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Figure 2.8: *Input/output tables for the addition of three binary digits, A, B, and C.*

numbers, you line them up one on top of the other, and add the digits in each column. In each column, there might also be a carry from the previous column. To add up a column, you only need to remember a small number of rules, such as $7 + 6 + 1 = 14$ and $3 + 5 + 0 = 8$. For binary addition, it's even easier, since the only digits are 0 and 1. There are only eight rules:

$$0 + 0 + 0 = 00 \qquad 1 + 0 + 0 = 01$$
$$0 + 0 + 1 = 01 \qquad 1 + 0 + 1 = 10$$
$$0 + 1 + 0 = 01 \qquad 1 + 1 + 0 = 10$$
$$0 + 1 + 1 = 10 \qquad 1 + 1 + 1 = 11$$

Here, we've written each sum using two digits. In a multi-column addition, one of these digits is carried over to the next column. Here, we have a calculation that has three inputs and two outputs. We can make an input/output table for each of the two outputs. The tables are shown in Figure 2.8. We know that these tables can be implemented as combinatorial circuits, so we know that circuits can add binary numbers. To add multi-digit binary numbers, we just need one copy of the basic addition circuit for each column in the sum.

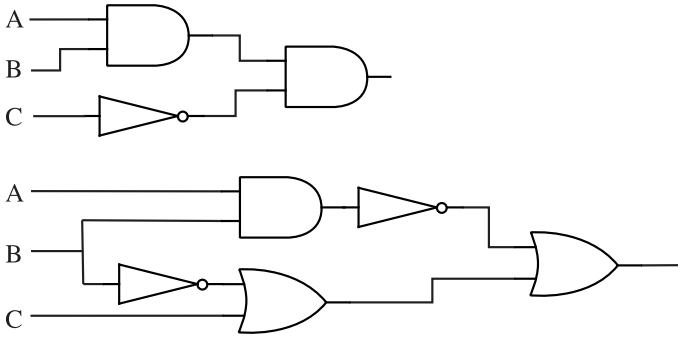
Exercises

1. Using only AND, OR, and NOT gates, draw circuits that compute the value of each of the propositions $A \rightarrow B$, $A \leftrightarrow B$, and $A \oplus B$.

2. For each of the following propositions, find a combinatorial logic circuit that computes that proposition:

a) $A \wedge (B \vee \neg C)$ **b)** $(p \wedge q) \wedge \neg(p \wedge \neg q)$
c) $(p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$ **d)** $\neg(A \wedge (B \vee C)) \vee (B \wedge \neg A)$

3. Find the compound proposition computed by each of the following circuits:

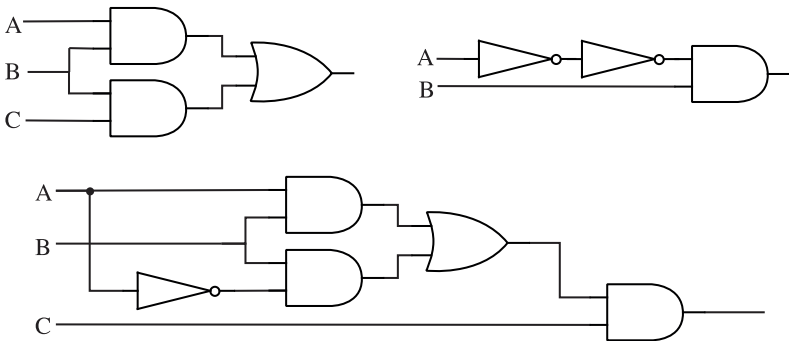


4. This section describes a method for finding the compound proposition computed by any combinatorial logic circuit. This method fails if you try to apply it to a circuit that contains a feedback loop. What goes wrong? Give an example.

5. Show that every compound proposition which is not a contradiction is equivalent to a proposition in disjunctive normal form. (Note: We can eliminate the restriction that the compound proposition is not a contradiction by agreeing that '$\mathbb{F}$' counts as a proposition in disjunctive normal form. $\mathbb{F}$ is logically equivalent to any contradiction.)

6. A proposition in ***conjunctive normal form*** (CNF) is a conjunction of disjunctions of simple terms (with the proviso, as in the definition of DNF that a single item counts as a conjunction or disjunction). Show that every compound proposition which is not a tautology is logically equivalent to a compound proposition in conjunctive normal form. (Hint: What happens if you take the negation of a DNF proposition and apply DeMorgan's Laws?)

7. Use the laws of Boolean algebra to simplify each of the following circuits:



8. Design circuits to implement the input/output tables for addition, as given in Figure 2.8. Try to make your circuits as simple as possible. (The circuits that are

used in real computers for this purpose are more simplified than the ones you will probably come up with, but the general approach of using logic to design computer circuits is valid. If you are interested to learn more about this, the second year variant course *Digital Systems* describes circuit design in more detail.)

2.4 Predicate Logic

In propositional logic, we can let p stand for "Roses are red" and q stand for "Violets are blue". Then $p \wedge q$ will stand for "Roses are red and violets are blue". But we lose a lot in the translation into logic. Since propositional logic only deals with truth values, there's nothing we can do with p and q in propositional logic that has anything to do with roses, violets, or colour. To apply logic to such things, we need ***predicates***. The type of logic that uses predicates is called ***predicate logic*** or, when the emphasis is on manipulating and reasoning with predicates, ***predicate calculus***.

2.4.1 Predicates

A predicate is a kind of incomplete proposition, which becomes a proposition when it is applied to some entity (or, as we'll see later, to several entities). In the proposition "the rose is red", the predicate is *is red*. By itself, 'is red' is not a proposition. Think of it as having an empty slot, that needs to be filled in to make a proposition: "— is red". In the proposition "the rose is red", the slot is filled by the entity "the rose", but it could just as well be filled by other entities: "the barn is red"; "the wine is red"; "the banana is red". Each of these propositions uses the same predicate, but they are different propositions and they can have different truth values.

If P is a predicate and a is an entity, then $P(a)$ stands for the proposition that is formed when P is applied to a. If P represents 'is red' and a stands for 'the rose', then $P(a)$ is 'the rose is red'. If M is the predicate 'is mortal' and s is 'Socrates', then $M(s)$ is the proposition "Socrates is mortal".

Now, you might be asking, just what is an *entity* anyway? I am using the term here to mean some specific, identifiable thing to which a predicate can be applied. Generally, it doesn't make sense to apply a given predicate to every possible entity, but only to entities in a certain category. For example, it probably doesn't make sense to apply the predicate 'is mortal' to your living room sofa. This predicate only applies to entities in the category of living things, since there is no way something can be mortal unless it is alive. This category is called the domain of discourse for the predicate.[9]

[9] In the language of set theory, which will be introduced in Chapter 4, we would say that a domain of discourse is a set, U, and a predicate is a function from U to the set of truth values.

We are now ready for a formal definition of one-place predicates. A one-place predicate, like all the examples we have seen so far, has a single slot which can be filled in with one entity:

Definition 2.7. A ***one-place predicate*** associates a proposition with each entity in some collection of entities. This collection is called the ***domain of discourse*** for the predicate. If P is a predicate and a is an entity in the domain of discourse for P, then $P(a)$ denotes the proposition that is associated with a by P. We say that $P(a)$ is the result of ***applying*** P to a.

We can obviously extend this to predicates that can be applied to two or more entities. In the proposition "John loves Mary", *loves* is a two-place predicate. Besides John and Mary, it could be applied to other pairs of entities: "John loves Jane", "Bill loves Mary", "John loves Bill", "John loves John". If Q is a two-place predicate, then $Q(a, b)$ denotes the proposition that is obtained when Q is applied to the entities a and b. Note that each of the 'slots' in a two-place predicate can have its own domain of discourse. For example, if Q represents the predicate 'owns', then $Q(a, b)$ will only make sense when a is a person and b is an inanimate object. An example of a three-place predicate is "a gave b to c", and a four-place predicate would be "a bought b from c for d euros". But keep in mind that not every predicate has to correspond to an English sentence.

When predicates are applied to entities, the results are propositions, and all the operators of propositional logic can be applied to these propositions just as they can to any propositions. Let R be the predicate 'is red', and let L be the two-place predicate 'loves'. If a, b, j, m, and b are entities belonging to the appropriate categories, then we can form compound propositions such as:

$R(a) \wedge R(b)$	a is red and b is red
$\neg R(a)$	a is not red
$L(j, m) \wedge \neg L(m, j)$	j loves m, and m does not love j
$L(j, m) \rightarrow L(b, m)$	if j loves m then b loves m
$R(a) \leftrightarrow L(j, j)$	a is red if and only if j loves j

The definition should be clear enough without the formal language of set theory, and in fact you should think of this definition—and many others—as motivation for that language.



Predicate logic is founded on the ideas developed by Charles Sanders Peirce (1839–1914), an American philosopher, logician, mathematician, and scientist.[10] Many of his contributions to logic were appreciated only years after he died. He has been called "the most original and versatile of American philosophers and America's greatest logician." and "one of the greatest philosophers ever". As early as 1886 he saw that logical operations could be carried out by electrical switching circuits; the same idea was used decades later to produce digital computers, as we saw in Section 2.3. You can read about his colourful life at the link below.

Source: `en.wikipedia.org/wiki/Charles_Sanders_Peirce`.

2.4.2 Quantifiers

Let's go back to the proposition with which we started this section: "Roses are red". This sentence is more difficult to handle than it might appear. We still can't express it properly in logic. The problem is that this proposition is not saying something about some particular entity. It really says that *all* roses are red (which happens to be a false statement, but that's what it means). Predicates can only be applied to individual entities.

Many other sentences raise similar difficulties: "All persons are mortal." "Some roses are red, but no roses are black." "All maths courses are interesting." "Every prime number greater than two is odd." Words like *all*, *no*, *some*, and *every* are called ***quantifiers***. We need to be able to express similar concepts in logic.

Suppose that P is a predicate, and we want to express the proposition that P is true when applied to any entity in the domain of discourse. That is, we want to say "for any entity x in the domain of discourse, $P(x)$ is true". In predicate logic, we write this in symbols as $\forall x(P(x))$. The $\forall$ symbol, which looks like an upside-down 'A', is usually read 'for all', so that $\forall x(P(x))$ is read as 'for all x, $P(x)$'. (It is understood that this means for all x in the domain of discourse for P.) For example, if R is the predicate 'is red' and the domain of discourse consists of all roses, then $\forall x(R(x))$ expresses the proposition "All roses are red". Note that the same proposition could be expressed in English as "Every rose is red" or "Any rose is red".

Now, suppose we want to say that a predicate, P, is true for *some* entity in its domain of discourse. This is expressed in predicate logic as $\exists x(P(x))$.

The $\exists$ symbol, which looks like a backwards 'E', is usually read 'there exists', but a more exact reading would be 'there is at least one'. Thus, $\exists x(P(x))$ is read as 'There exists an x such that $P(x)$' , and it means "there is at least one x in the domain of discourse for P for which $P(x)$ is true". If, once again, R stands for 'is red' and the domain of discourse is 'roses', then $\exists x(R(x))$ could be expressed in English as "There is a red rose" or "At least one rose is red" or "Some rose is red". It might also be expressed as "Some roses are red", but the plural is a bit misleading since $\exists x(R(x))$ is true even if there is only one red rose. We can now give the formal definitions:

Definition 2.8. Suppose that P is a one-place predicate. Then $\forall x(P(x))$ is a proposition, which is true if and only if $P(a)$ is true for every entity a in the domain of discourse for P. And $\exists x(P(x))$ is a proposition which is true if and only if there is at least one entity, a, in the domain of discourse for P for which $P(a)$ is true. The $\forall$ symbol is called the ***universal quantifier***, and $\exists$ is called the ***existential quantifier***.

The x in $\forall x(P(x))$ and $\exists x(P(x))$ is a variable. (More precisely, it is an *entity* variable, since its value can only be an entity.) Note that a plain $P(x)$—without the $\forall x$ or $\exists x$—is not a proposition. $P(x)$ is neither true nor false because x is not some particular entity, but just a placeholder in a slot that can be filled in with an entity. $P(x)$ would stand for something like the statement 'x is red', which is not really a statement in English at all. But it becomes a statement when the x is replaced by some particular entity, such as 'the rose'. Similarly, $P(x)$ becomes a proposition if some entity a is substituted for the x, giving $P(a)$.[11]

An ***open statement*** is an expression that contains one or more entity variables, which becomes a proposition when entities are substituted for the variables. (An open statement has open 'slots' that need to be filled in.) $P(x)$ and "x is red" are examples of open statements that contain one variable. If L is a two-place predicate and x and y are variables, then $L(x, y)$ is an open statement containing two variables. An example in English would be "x loves y". The variables in an open statement are called ***free variables***. An open statement that contains x as a free variable can be quantified with $\forall x$ or $\exists x$. The variable x is then said to be ***bound***. For example, x is free in $P(x)$ and is bound in $\forall x(P(x))$ and $\exists x(P(x))$. The free variable y in $L(x, y)$ becomes bound in $\forall y(L(x, y))$ and in $\exists y(L(x, y))$.

Note that $\forall y(L(x, y))$ is still an open statement, since it contains x as a free variable. Therefore, it is possible to apply the quantifier $\forall x$ or $\exists x$ to

[11] There is certainly room for confusion about names here. In this discussion, x is a variable and a is an entity. But that's only because we said so. Any letter could be used in either role, and you have to pay attention to the context to figure out what is going on. Usually, x, y, and z will be variables.

$\forall y(L(x,y))$, giving $\forall x\big(\forall y(L(x,y))\big)$ and $\exists x\big(\forall y(L(x,y))\big)$. Since all the variables are bound in these expressions, they are propositions. If $L(x,y)$ represents 'x loves y', then $\forall y(L(x,y))$ is something like "x loves everyone", and $\exists x\big(\forall y(L(x,y))\big)$ is the proposition, "There is someone who loves everyone". Of course, we could also have started with $\exists x(L(x,y))$: "There is someone who loves y". Applying $\forall y$ to this gives $\forall y\big(\exists x(L(x,y))\big)$, which means "For every person, there is someone who loves that person". Note in particular that $\exists x\big(\forall y(L(x,y))\big)$ and $\forall y\big(\exists x(L(x,y))\big)$ do *not* mean the same thing. Altogether, there are eight different propositions that can be obtained from $L(x,y)$ by applying quantifiers, with six distinct meanings among them.



From now on, we will leave out parentheses when there is no ambiguity. For example, we will write $\forall x\, P(x)$ instead of $\forall x(P(x))$ and $\exists x\, \exists y\, L(x,y)$ instead of $\exists x\big(\exists y(L(x,y))\big)$. Make sure though that when you leave out the parentheses you do so only when no ambiguity exists. In one of the problems of this chapter, you will see an example of two very similar statements where the parentheses do change the meaning significantly!

Further, we will sometimes give predicates and entities names that are complete words instead of just letters, as in $Red(x)$ and $Loves(john, mary)$. This might help to make examples more readable.

2.4.3 Operators

In predicate logic, the operators and laws of Boolean algebra still apply. For example, if P and Q are one-place predicates and a is an entity in the domain of discourse, then $P(a) \to Q(a)$ is a proposition, and it is logically equivalent to $\neg P(a) \vee Q(a)$. Further, if x is a variable, then $P(x) \to Q(x)$ is an open statement, and $\forall x(P(x) \to Q(x))$ is a proposition. So are $P(a) \wedge (\exists x\, Q(x))$ and $(\forall x\, P(x)) \to (\exists x P(x))$. Obviously, predicate logic can be very expressive. Unfortunately, the translation between predicate logic and English sentences is not always obvious.



One of the commonly-made mistakes in predicate logic is the difference in translation between statements like: "All humans are mortal" and "There is a human that is mortal". We discuss the difference in translation of these statements in one of our pencasts: `youtu.be/BJeGHIX_ldY`.

Let's look one more time at the proposition "Roses are red". If the domain of discourse consists of roses, this translates into predicate logic as $\forall x\, Red(x)$. However, the sentence makes more sense if the domain of discourse is larger—for example if it consists of all flowers. Then "Roses are red" has to be read as "All flowers which are roses are red", or "For any flower, if that flower is a rose, then it is red". The last form translates directly into logic as $\forall x\big(Rose(x) \to Red(x)\big)$. Suppose we want to say that all red roses are pretty. The phrase 'red rose' is saying both that the flower is a rose and that it is red, and it must be translated as a conjunction, $Rose(x) \wedge Red(x)$. So, "All red roses are pretty" can be rendered as $\forall x\big((Rose(x) \wedge Red(x)) \to Pretty(x)\big)$.



Here are a few more examples of translations from predicate logic to English. Let $H(x)$ represent 'x is happy', let $C(y)$ represent 'y is a computer', and let $O(x, y)$ represent 'x owns y'. Then we have the following translations:

- Jack owns a computer: $\exists x\big(O(jack, x) \wedge C(x)\big)$. (That is, there is at least one thing such that Jack owns that thing and that thing is a computer.)
- Everything Jack owns is a computer: $\forall x\big(O(jack, x) \rightarrow C(x)\big)$.
- If Jack owns a computer, then he's happy:
  $$\big(\exists y(O(jack, y) \wedge C(y))\big) \rightarrow H(jack).$$
- Everyone who owns a computer is happy:
  $$\forall x\big(\,\big(\exists y(O(x, y) \wedge C(y))\big) \rightarrow H(x)\big)\,\big).$$
- Everyone owns a computer: $\forall x\,\exists y\big(C(y) \wedge O(x, y)\big)$. (Note that this allows each person to own a different computer. The proposition $\exists y\,\forall x\big(C(y) \wedge O(x, y)\big)$ would mean that there is a single computer which is owned by everyone.)
- Everyone is happy: $\forall x H(x)$.
- Everyone is unhappy: $\forall x(\neg H(x))$.
- Someone is unhappy: $\exists x(\neg H(x))$.
- At least two people are happy: $\exists x \exists y\big(H(x) \wedge H(y) \wedge (x \neq y)\big)$. (The stipulation that $x \neq y$ is necessary because two different variables can refer to the same entity. The proposition $\exists x \exists y(H(x) \wedge H(y))$ is true even if there is only one happy person.)
- There is exactly one happy person:
  $$\big(\exists x H(x))\big) \wedge \big(\forall y \forall z((H(y) \wedge H(z)) \rightarrow (y = z))\big).$$
  (The first part of this conjunction says that there is at least one happy person. The second part says that if y and z are both happy people, then they are actually the same person. That is, it's not possible to find two *different* people who are happy. The statement can be simplified a little however, to get: $\exists x(H(x) \wedge \forall y(H(y) \rightarrow (x = y)))$. Do you see why this works as well?)
- For another worked example, check out the pencast on the topic here: `https://youtu.be/XsI2DJpaGYA`

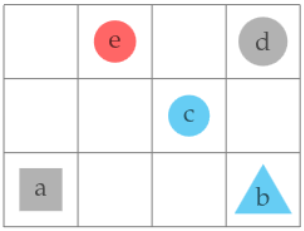


Figure 2.9: *An instance of a Tarski World.*

2.4.4 Tarski's world and formal structures

To help you reason about sets of predicate logic statements, or even arguments expressed in predicate logic, we often use a 'mathematical structure'. For some of these structures a visualisation in the form of a ***Tarski's world*** can sometimes be useful.





What is truth? In 1933, Polish mathematician Alfred Tarski (1901–1983) published a very long paper in Polish (titled *Pojęcie prawdy w językach nauk dedukcyjnych*), setting out a mathematical definition of truth for formal languages. "Along with his contemporary, Kurt Gödel [who we'll see in Chapter 4], he changed the face of logic in the twentieth century, especially through his work on the concept of truth and the theory of models."

Source: `en.wikipedia.org/wiki/Alfred_Tarski`.

In a Tarski's world, it is possible to describe situations using formulas whose truth can be evaluated, which are expressed in a first-order language that uses predicates such as Rightof(x, y), which means that x is situated—somewhere, not necessarily directly—to the right of y, or Blue(x), which means that x is blue. In the world in Figure 2.9, for instance, the formula $\forall x(\text{Triangle}(x) \rightarrow \text{Blue}(x))$ holds, since all triangles are blue, but the converse of this formula, $\forall x(\text{Blue}(x) \rightarrow \text{Triangle}(x))$, does not hold, since object c is blue but not a triangle.

Such an instance of Tarski world can be more formally described as a

'mathematical structure' (which we refer to as a ***formal structure*** occasionally). These structures allow us to evaluate statements in predicate logic as being true or false. To formalise a structure, we need to describe two things: the domain of discourse D of the structure and for all of the predicates, for which objects of the domain they are true. We do so using *set-notation* which we discuss in more depth in Chapter 4. The formal description of the structure S depicted in Figure 2.9 is:

- $D = \{a, b, c, d, e\}$
- $Blue^S = \{b, c\}$
- $Gray^S = \{a, d\}$
- $Red^S = \{e\}$
- $Square^S = \{a\}$
- $Triangle^S = \{b\}$
- $Circle^S = \{c, d, e\}$
- $RightOf^S = \{(b,a), (c,a), (d,a), (e,a), (b,c), (d,c), (b,e), (c,e), (d,e)\}$
- $LeftOf^S = \{(a,b), (c,b), (e,b), (a,c), (e,c), (a,d), (c,d), (e,d), (a,e)\}$
- $BelowOf^S = \{(a,c), (a,d), (a,e), (b,c), (b,d), (b,e), (c,d), (c,e)\}$
- $AboveOf^S = \{(c,a), (c,b), (d,a), (d,b), (d,c), (e,a), (e,b), (e,c)\}$

Notice that for the one-place predicates we have a set of objects for which this predicate is true (e.g., only b and c are blue) and such a set is denoted using '{' and '}' symbols, called 'curly braces' or just 'braces'.[12] For the two-place predicates we have a set of ***tuples*** that are denoted using '(' and ')' symbols, called 'parentheses' or 'round brackets'. In this case, for instance, the fact that (a, b) is in the set $LeftOf^S$ means that $LeftOf(a, b)$ is true for this structure, i.e., a is left of b.

Such formal structures can also be defined to disprove arguments written in predicate logic, as we will see in Section 2.5.3.

2.4.5 Logical equivalence

To calculate in predicate logic, we need a notion of logical equivalence. Clearly, there are pairs of propositions in predicate logic that mean the same thing. Consider the propositions $\neg(\forall x H(x))$ and $\exists x(\neg H(x))$, where $H(x)$ represents 'x is happy'. The first of these propositions means "Not everyone is happy", and the second means "Someone is not happy". These statements have the same truth value: if not everyone is happy, then someone is unhappy and vice versa. But logical equivalence is much stronger than just having the same truth value. In propositional logic, logical equivalence is defined in terms of propositional variables: two compound propositions are logically equivalent if they have the same truth values for all possible truth values of the propositional variables they contain. In predicate logic,

[12] See Chapter 4.

$\neg\,(\forall x P(x)) \equiv \exists x(\neg P(x))$
$\neg\,(\exists x P(x)) \equiv \forall x(\neg P(x))$
$\forall x \forall y Q(x,y) \equiv \forall y \forall x Q(x,y)$
$\exists x \exists y Q(x,y) \equiv \exists y \exists x Q(x,y)$

Figure 2.10: *Four important rules of predicate logic. P can be any one-place predicate, and Q can be any two-place predicate. The first two rules are called DeMorgan's Laws for predicate logic.*

two formulas are logically equivalent if they have the same truth value for all possible predicates.

Consider $\neg(\forall x P(x))$ and $\exists x(\neg P(x))$. These formulas make sense for any predicate P, and for any predicate P they have the same truth value. Unfortunately, we can't—as we did in propositional logic—just check this fact with a truth table: there are no subpropositions, connected by $\wedge$, $\vee$, etc, out of which to build a table. So, let's reason it out: To say $\neg(\forall x P(x))$ is true is just to say that it is not the case that $P(x)$ is true for all possible entities x. So, there must be some entity a for which $P(a)$ is false. Since $P(a)$ is false, $\neg P(a)$ is true. But saying that there is an a for which $\neg P(a)$ is true is just saying that $\exists x(\neg P(x))$ is true. So, the truth of $\neg(\forall x P(x))$ implies the truth of $\exists x(\neg P(x))$. On the other hand, if $\neg(\forall x P(x))$ is false, then $\forall x P(x)$ is true. Since $P(x)$ is true for every x, $\neg P(x)$ is false for every x; that is, there is no entity a for which the statement $\neg P(a)$ is true. But this just means that the statement $\exists x(\neg P(x))$ is false. In any case, then, the truth values of $\neg(\forall x P(x))$ and $\exists x(\neg P(x))$ are the same. Since this is true for any predicate P, we will say that these two formulas are logically equivalent and write $\neg(\forall x P(x)) \equiv \exists x(\neg P(x))$.

A similar argument would show that $\neg(\exists x P(x)) \equiv \forall x(\neg P(x))$. These two equivalences, which explicate the relation between negation and quantification, are known as DeMorgan's Laws for predicate logic. (They are closely related to DeMorgan's Laws for propositional logic; see the exercises.) These laws can be used to help simplify expressions. For example,

$$\begin{aligned} \neg\,\forall y(R(y) \vee Q(y)) &\equiv \exists y(\neg(R(y) \vee Q(y))) \\ &\equiv \exists y((\neg R(y)) \wedge (\neg Q(y)) \end{aligned}$$

It might not be clear exactly why this qualifies as a 'simplification', but it's generally considered simpler to have the negation operator applied to basic propositions such as $R(y)$, rather than to quantified expressions such as

$\forall y(R(y) \vee Q(y))$. For a more complicated example:

$$\begin{aligned}
&\neg\,\exists x\big(P(x) \wedge (\forall y(Q(y) \to Q(x)))\big) \\
&\qquad \equiv \forall x\big(\neg\big(P(x) \wedge (\forall y(Q(y) \to Q(x)))\big)\big) \\
&\qquad \equiv \forall x\big((\neg P(x)) \vee (\neg\forall y(Q(y) \to Q(x)))\big) \\
&\qquad \equiv \forall x\big((\neg P(x)) \vee (\exists y(\neg(Q(y) \to Q(x))))\big) \\
&\qquad \equiv \forall x\big((\neg P(x)) \vee (\exists y(\neg(\neg Q(y) \vee Q(x))))\big) \\
&\qquad \equiv \forall x\big((\neg P(x)) \vee (\exists y(\neg\neg Q(y) \wedge \neg Q(x)))\big) \\
&\qquad \equiv \forall x\big((\neg P(x)) \vee (\exists y(Q(y) \wedge \neg Q(x)))\big)
\end{aligned}$$

DeMorgan's Laws are listed in Figure 2.10 along with two other laws of predicate logic. The other laws allow you to interchange the order of the variables when two quantifiers of the same type (both $\exists$ or $\forall$) occur together.



Notice however that we may not change the order of quantifiers that are not the same! For instance: $\forall x\exists y(R(x,y)) \not\equiv \exists y\forall x(R(x,y))$. If you are not convinced about this, try to draw up a Tarski's world that shows this unequivalence.

To define logical equivalence in predicate logic more formally, we need to talk about formulas that contain predicate variables, that is, variables that act as place-holders for arbitrary predicates in the same way that propositional variables are place-holders for propositions and entity variables are place-holders for entities. With this in mind, we can define logical equivalence and the closely related concept of tautology for predicate logic. We'll see that these are crucial pieces of writing proofs.

Definition 2.9. Let $\mathcal{P}$ be a formula of predicate logic which contains one or more predicate variables. $\mathcal{P}$ is said to be a ***tautology*** if it is true whenever all the predicate variables that it contains are replaced by actual predicates. Two formulas $\mathcal{P}$ and $\mathcal{Q}$ are said to be ***logically equivalent*** if $\mathcal{P} \leftrightarrow \mathcal{Q}$ is a tautology, that is if $\mathcal{P}$ and $\mathcal{Q}$ always have the same truth value when the predicate variables they contain are replaced by actual predicates. The notation $\mathcal{P} \equiv \mathcal{Q}$ asserts that $\mathcal{P}$ is logically equivalent to $\mathcal{Q}$.

Exercises

†**1.** Simplify each of the following propositions. In your answer, the $\neg$ operator should be applied only to individual predicates.

a) $\neg\,\forall x(\neg P(x))$ **b)** $\neg\,\exists x(P(x) \wedge Q(x))$

c) $\neg\,\forall z(P(z) \to Q(z))$ **d)** $\neg\big((\forall x P(x)) \wedge (\forall y Q(y))\big)$
e) $\neg\,\forall x \exists y P(x,y)$ **f)** $\neg\,\exists x(R(x) \wedge \forall y S(x,y))$
g) $\neg\,\exists y(P(y) \leftrightarrow Q(y))$ **h)** $\neg\big(\forall x(P(x) \to (\exists y Q(x,y)))\big)$

2. Give a careful argument to show that the second of DeMorgan's laws for predicate calculus, $\neg(\forall x P(x)) \equiv \exists x(\neg P(x))$, is valid.

3. Find the negation of each of the following propositions. Simplify the result; in your answer, the $\neg$ operator should be applied only to individual predicates.
a) $\exists n(\forall s C(s,n))$
b) $\exists n(\forall s(L(s,n) \to P(s)))$
c) $\exists n(\forall s(L(s,n) \to (\exists x \exists y \exists z Q(x,y,z))))$.
d) $\exists n(\forall s(L(s,n) \to (\exists x \exists y \exists z(s = xyz \wedge R(x,y) \wedge T(y) \wedge U(x,y,z)))))$.

4. Suppose that the domain of discourse for a predicate P contains only two entities. Show that $\forall x P(x)$ is equivalent to a conjunction of two simple propositions, and $\exists x P(x)$ is equivalent to a disjunction. Show that in this case, DeMorgan's Laws for propositional logic and DeMorgan's Laws for predicate logic actually say exactly the same thing. Extend the results to a domain of discourse that contains exactly three entities.

5. Let $H(x)$ stand for 'x is happy', where the domain of discourse consists of people. Express the proposition "There are exactly three happy people" in predicate logic.

6. What is the difference between the following two statements?
$\exists x Red(x) \wedge \exists x Square(x)$ and $\exists x(Red(x) \wedge Square(x))$

7. Draw a Tarski world for the last exercise.

†**8.** Express Johan Cruyff's statement "There is only one ball, so you need to have it" in predicate logic.

9. Let $T(x,y)$ stand for 'x has taken y', where the domain of discourse for x consists of students and the domain of discourse for y consists of CS courses (at TUDelft). Translate each of the following propositions into an unambiguous English sentence:
a) $\forall x\,\forall y\,T(x,y)$ **b)** $\forall x\,\exists y\,T(x,y)$ **c)** $\forall y\,\exists x\,T(x,y)$
d) $\exists x\,\exists y\,T(x,y)$ **e)** $\exists x\,\forall y\,T(x,y)$ **f)** $\exists y\,\forall x\,T(x,y)$

10. Let $F(x,t)$ stand for "You can fool person x at time t." Translate the following sentence into predicate logic: "You can fool some of the people all of the time, and you can fool all of the people some of the time, but you can't fool all of the people all of the time."

11. Translate each of the following sentences into a proposition using predicate logic. Make up any predicates you need. State clearly what each predicate means.
a) All crows are black.
b) Any white bird is not a crow.
c) Not all politicians are honest.
d) All purple elephants have green feet.
e) There is no one who does not like pizza.

f) Anyone who passes the final exam will pass the course.[13]

g) If x is any positive number, then there is a number y such that $y^2 = x$.

12. Consider the following description of a Tarski World. Does an instance of a Tarski World exist with these properties? If so, give one with a domain of at most 5 elements. If no such instance exists, explain why not.

- $\forall x(Circle(x) \rightarrow \neg Blue(x))$
- $\exists x(Circle(x)) \wedge \exists x(Blue(x))$
- $RightOf(a, b)$
- $LeftOf(a, b) \vee Square(c)$

†**13.** The sentence "Someone has the answer to every question" is ambiguous. Give two translations of this sentence into predicate logic, and explain the difference in meaning.

14. The sentence "Jane is looking for a dog" is ambiguous. One meaning is that there is some particular dog—maybe the one she lost—that Jane is looking for. The other meaning is that Jane is looking for any old dog—maybe because she wants to buy one. Express the first meaning in predicate logic. Explain why the second meaning is *not* expressed by $\forall x(Dog(x) \rightarrow LooksFor(jane, x))$. In fact, the second meaning cannot be expressed in predicate logic. Philosophers of language spend a lot of time thinking about things like this. They are especially fond of the sentence "Jane is looking for a unicorn", which is not ambiguous when applied to the real world. Why is that?

2.5 Deduction

Logic can be applied to draw conclusions from a set of ***premises***. A premise is just a proposition that is known to be true or that has been accepted to be true for the sake of argument, and a conclusion is a proposition that can be deduced logically from the premises. The idea is that if you believe that the premises are true, then logic forces you to accept that the conclusion is true. An ***argument*** is a claim that a certain conclusion follows from a given set of premises. Here is an argument laid out in a traditional format:

If today is Tuesday, then this is Belgium
Today is Tuesday

$\therefore$ This is Belgium

The premises of the argument are shown above the line, and the conclusion below. The symbol $\therefore$ is read 'therefore'. The claim is that the conclusion, "This is Belgium", can be deduced logically from the two premises, "If today is Tuesday, then this is Belgium" and "Today is Tuesday". In fact, this claim is true. Logic forces you to accept this argument. Why is that?

[13] This is not true for *Reasoning & Logic*: see the syllabus.

2.5.1 Arguments

Let p stand for the proposition "Today is Tuesday", and let q stand for the proposition "This is Belgium". Then the above argument has the form

$$\begin{array}{l} p \to q \\ p \\ \hline \therefore\ q \end{array}$$

Now, for *any* propositions p and q—not just the ones in this particular argument—if $p \to q$ is true and p is true, then q must also be true. This is easy to check in a truth table:

p	q	$p \to q$
0	0	1
0	1	1
1	0	0
1	1	1

The only case where both $p \to q$ and p are true is on the last line of the table, and in this case, q is also true. If you believe $p \to q$ and p, you have no logical choice but to believe q. This applies no matter what p and q represent. For example, if you believe "If Jill is breathing, then Jill pays taxes", and you believe that "Jill is breathing", logic forces you to believe that "Jill pays taxes". Note that we can't say for sure that the conclusion is true, only that *if* the premises are true, *then* the conclusion must be true.

This fact can be rephrased by saying that $\big((p \to q) \wedge p\big) \to q$ is a tautology. More generally, for any compound propositions $\mathcal{P}$ and $\mathcal{Q}$, saying "$\mathcal{P} \to \mathcal{Q}$ is a tautology" is the same as saying that "in all cases where $\mathcal{P}$ is true, $\mathcal{Q}$ is also true".[14] We will use the notation $\mathcal{P} \Longrightarrow \mathcal{Q}$ to mean that $\mathcal{P} \to \mathcal{Q}$ is a tautology. Think of $\mathcal{P}$ as being the premise of an argument or the conjunction of several premises. To say $\mathcal{P} \Longrightarrow \mathcal{Q}$ is to say that $\mathcal{Q}$ follows logically from $\mathcal{P}$. We will use the same notation in both propositional logic and predicate logic. (Note that the relation of $\Longrightarrow$ to $\to$ is the same as the relation of $\equiv$ to $\leftrightarrow$.)

Definition 2.10. Let $\mathcal{P}$ and $\mathcal{Q}$ be any formulas in either propositional logic or predicate logic. The notation $\mathcal{P} \Longrightarrow \mathcal{Q}$ is used to mean that $\mathcal{P} \to \mathcal{Q}$ is a tautology. That is, in all cases where $\mathcal{P}$ is true, $\mathcal{Q}$ is also true. We then say that $\mathcal{Q}$ can be ***logically deduced*** from $\mathcal{P}$ or that $\mathcal{P}$ ***logically implies*** $\mathcal{Q}$.

[14] Here, "in all cases" means for all combinations of truth values of the propositional variables in $\mathcal{P}$ and $\mathcal{Q}$, i.e., in every situation. Saying $\mathcal{P} \to \mathcal{Q}$ is a tautology means it is true in all cases. But by definition of $\to$, it is automatically true in cases where $\mathcal{P}$ is false. In cases where $\mathcal{P}$ is true, $\mathcal{P} \to \mathcal{Q}$ will be true if and only if $\mathcal{Q}$ is true.

An argument in which the conclusion follows logically from the premises is said to be a ***valid argument***. To test whether an argument is valid, you have to replace the particular propositions or predicates that it contains with variables, and then test whether the conjunction of the premises logically implies the conclusion. We have seen that any argument of the form

$$\begin{array}{l} p \rightarrow q \\ \underline{p} \\ \therefore\ q \end{array}$$

is valid, since $\big((p \rightarrow q) \wedge p\big) \rightarrow q$ is a tautology. This rule of deduction is called ***modus ponens***. It plays a central role in logic. Another, closely related rule is ***modus tollens***, which applies to arguments of the form

$$\begin{array}{l} p \rightarrow q \\ \underline{\neg q} \\ \therefore\ \neg p \end{array}$$

To verify that this is a valid argument, just check that $\big((p \rightarrow q) \wedge \neg q\big) \Longrightarrow \neg p$, that is, that $\big((p \rightarrow q) \wedge \neg q\big) \rightarrow \neg p$ is a tautology. As an example, the following argument has the form of *modus tollens* and is therefore a valid argument:

If Feyenoord is a great team, then I'm the king of the Netherlands
I am not the king of the Netherlands
$\therefore$ Feyenoord is not a great team

You might remember this argument from page 13. You should note carefully that the validity of this argument has nothing to do with whether or not Feyenoord can play football well. The argument forces you to accept the conclusion *only if* you accept the premises. You can logically believe that the conclusion is false, as long as you believe that at least one of the premises is false.[15]

Another named rule of deduction is the ***Law of Syllogism***, which has the form

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore\ p \rightarrow r \end{array}$$

For example:

[15]Unless the conclusion is a tautology. If that's the case, then even when a premise is false the conclusion will still be true. You do always know that if the conclusion is false then at least one of the premises is false.

If you study hard, you do well in school
If you do well in school, you get a good job
$\therefore$ If you study hard, you get a good job

There are many other rules. Here are a few that might prove useful. Some of them might look trivial, but don't underestimate the power of a simple rule when it is combined with other rules.

$$\frac{p \vee q \quad \neg p}{\therefore\ q} \qquad \frac{p \quad q}{\therefore\ p \wedge q} \qquad \frac{p \wedge q}{\therefore\ p} \qquad \frac{p}{\therefore\ p \vee q}$$

Logical deduction is related to logical equivalence. We defined $\mathcal{P}$ and $\mathcal{Q}$ to be logically equivalent if $\mathcal{P} \leftrightarrow \mathcal{Q}$ is a tautology. Since $\mathcal{P} \leftrightarrow \mathcal{Q}$ is equivalent to $(\mathcal{P} \rightarrow \mathcal{Q}) \wedge (\mathcal{Q} \rightarrow \mathcal{P})$, we see that $\mathcal{P} \equiv \mathcal{Q}$ if and only if both $\mathcal{Q} \implies \mathcal{P}$ and $\mathcal{P} \implies \mathcal{Q}$. Thus, we can show that two statements are logically equivalent if we can show that each of them can be logically deduced from the other. Also, we get a lot of rules about logical deduction for free—two rules of deduction for each logical equivalence we know. For example, since $\neg(p \wedge q) \equiv (\neg p \vee \neg q)$, we get that $\neg(p \wedge q) \implies (\neg p \vee \neg q)$. For example, if we know "It is not both sunny and warm", then we can logically deduce "Either it's not sunny or it's not warm." (And vice versa.)

2.5.2 Valid arguments and proofs

In general, arguments are more complicated than those we've considered so far. Here, for example, is an argument that has five premises:

$$\frac{\begin{array}{l}(p \wedge r) \rightarrow s \\ q \rightarrow p \\ t \rightarrow r \\ q \\ t\end{array}}{\therefore\ s}$$

Is this argument valid? Of course, you could use a truth table to check whether the conjunction of the premises logically implies the conclusion. But with five propositional variables, the table would have 32 lines, and the size of the table grows quickly when more propositional variables are used. So, in general, truth tables are not practical when we have a large number of variables.



For a relatively small number of variables (say three or fewer) a truth table can be a rather efficient method to test validity of an argument. In one of the pencasts of this course I show how you can use truth tables to test for validity as well as how you can use them to find counterexamples for invalid arguments: `youtu.be/lSZS3qbA88o`

Fortunately, there is another way to proceed, based on the fact that it is possible to chain several logical deductions together. That is, if $\mathcal{P} \implies \mathcal{Q}$ and $\mathcal{Q} \implies \mathcal{R}$, it follows that $\mathcal{P} \implies \mathcal{R}$. This means we can demonstrate the validity of an argument by deducing the conclusion from the premises in a sequence of steps. These steps can be presented in the form of a proof:

Definition 2.11. A ***formal proof*** that an argument is valid consists of a sequence of propositions such that the last proposition in the sequence is the conclusion of the argument, and every proposition in the sequence is either a premise of the argument or follows by logical deduction from propositions that precede it in the list.

The existence of such a proof shows that the conclusion follows logically from the premises, and therefore that the argument is valid. Here is a formal proof that the argument given above is valid. The propositions in the proof are numbered, and each proposition has a justification.

Proof.

1. $q \to p$	premise
2. q	premise
3. p	from 1 and 2 (*modus ponens*)
4. $t \to r$	premise
5. t	premise
6. r	from 4 and 5 (*modus ponens*)
7. $p \wedge r$	from 3 and 6
8. $(p \wedge r) \to s$	premise
9. s	from 7 and 8 (*modus ponens*)

□



Once a formal proof has been constructed, it is convincing. Unfortunately, it's not necessarily easy to come up with the proof. Usually, the best method is a combination of working forward ("Here's what I know, what can I deduce from that?") and working backwards ("Here's what I need to prove, what other things would imply that?"). For this proof, I might have thought: I want to prove s. I know that $p \wedge r$ implies s, so if I can prove $p \wedge r$, I'm okay. But to prove $p \wedge r$, it'll be enough to prove p and r separately....

Of course, not every argument is valid, so the question also arises, how can we show that an argument is invalid? Let's assume that the argument has been put into general form, with all the specific propositions replaced by propositional variables. The argument is valid if in all cases where all the premises are true, the conclusion is also true. The argument is invalid if there is even one case where all the premises are true and the conclusion is false. We can prove that an argument is invalid by finding an assignment of truth values to the propositional variables which makes all the premises true but makes the conclusion false. We call such an assignment a *counterexample*. To disprove the validity of an argument you should always provide a counterexample. This holds in propositional logic, predicate logic, and any other type of argument you may be asked to disprove.

For example, consider an argument of the form:

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow (p \wedge r) \\ r \\ \hline \therefore\ p \end{array}$$

In the case where p is false, q is false, and r is true, the three premises of this argument are all true, but the conclusion is false. This counterexample shows that the argument is invalid.

To apply all this to arguments stated in English, we have to introduce propositional variables to represent all the propositions in the argument. For example, consider:

> John will be at the party if Mary is there and Bill is not there. Mary will be at the party if it's on Friday or Saturday. If Bill is at the party, Tom will be there. Tom won't be at the party if it's on Friday. The party is on Friday. Therefore, John will be at the party.

Let j stand for "John will be at the party", m for "Mary will be there", b for "Bill will be there", t for "Tom will be there", f for "The party is on Friday", and s for "The party is on Saturday". Then this argument has the form

$$\begin{array}{l} (m \wedge \neg b) \rightarrow j \\ (f \vee s) \rightarrow m \\ b \rightarrow t \\ f \rightarrow \neg t \\ \underline{f} \\ \therefore\ j \end{array}$$

This is a valid argument, as the following proof shows:

Proof.

1.	$f \rightarrow \neg t$	premise
2.	f	premise
3.	$\neg t$	from 1 and 2 (*modus ponens*)
4.	$b \rightarrow t$	premise
5.	$\neg b$	from 4 and 3 (*modus tollens*)
6.	$f \vee s$	from 2
7.	$(f \vee s) \rightarrow m$	premise
8.	m	from 6 and 7 (*modus ponens*)
9.	$m \wedge \neg b$	from 8 and 5
10.	$(m \wedge \neg b) \rightarrow j$	premise
11.	j	from 10 and 9 (*modus ponens*)

□



You may have noticed that we start our proofs with the word 'proof' and end it with a little square. This is done to illustrate clearly where our proof starts and ends. Historically different symbols and expressions have been used to indicate that a proof is done. You may have heard of the abbreviation Q.E.D. for instance for 'Quod Erat Demonstrandum', which translates to: 'what was to be shown'. Even in ancient Greece a Greek version of Q.E.D. was used by Greek mathematicians like Euclid. You are free to choose between Q.E.D. and the open square, so long as you remember that no proof is complete if it does not have either one of them.

2.5.3 Proofs in predicate logic

So far in this section, we have been working mostly with propositional logic. But the definitions of valid argument and logical deduction apply to predicate logic as well.

One of the most basic rules of deduction in predicate logic says that $(\forall x P(x)) \implies P(a)$ for any entity a in the domain of discourse of the predicate P. That is, if a predicate is true of all entities, then it is true of any given particular entity. This rule can be combined with rules of deduction for propositional logic to give the following valid arguments:

$$\frac{\begin{array}{l}\forall x(P(x) \to Q(x)) \\ P(a)\end{array}}{\therefore\ Q(a)} \qquad \frac{\begin{array}{l}\forall x(P(x) \to Q(x)) \\ \neg Q(a)\end{array}}{\therefore\ \neg P(a)}$$

These valid arguments go by the names of *modus ponens* and *modus tollens* for predicate logic. Note that from the premise $\forall x(P(x) \to Q(x))$ we can deduce $P(a) \to Q(a)$. From this and from the premise that $P(a)$, we can deduce $Q(a)$ by *modus ponens*. So the first argument above is valid. The second argument is similar, using *modus tollens*.

The most famous logical deduction of them all is an application of *modus ponens* for predicate logic:

$$\frac{\begin{array}{l}\text{All humans are mortal} \\ \text{Socrates is human}\end{array}}{\therefore\ \text{Socrates is mortal}}$$

This has the form of *modus ponens* with $P(x)$ standing for "x is human", $Q(x)$ standing for "x is mortal", and a standing for the noted entity, Socrates.

To disprove validity of arguments in predicate logic, you again need to provide a counterexample. These are most easily given in the form of a mathematical structure. Consider for instance the following argument:

$$\frac{\begin{array}{l}\exists x P(x) \\ \forall x(P(x) \to Q(x))\end{array}}{\therefore\ \forall x Q(x)}$$

This argument is not valid and we can prove that using the following structure $\mathcal{A}$.

- $D = \{a, b\}$
- $P^{\mathcal{A}} = \{a\}$
- $Q^{\mathcal{A}} = \{a\}$

As you can see, the first premise is true. There is an x such that $P(x)$ holds, namely $x = a$. The second premise is also true, as for all x for which $P(x)$ holds (so only $x = a$), $Q(x)$ also holds (and indeed $Q(a)$) holds. However the conclusion is false, as $Q(b)$ does not hold, so the $Q(x)$ does not hold for all x.

There is a lot more to say about logical deduction and proof in predicate logic, and we'll spend the whole of the next chapter on the subject.

Exercises

†**1.** Verify the validity of *modus tollens* and the Law of Syllogism.

†**2.** Each of the following is a valid rule of deduction. For each one, give an example of a valid argument in English that uses that rule.

$$\frac{p \vee q \quad \neg p}{\therefore\ q} \qquad \frac{p \quad q}{\therefore\ p \wedge q} \qquad \frac{p \wedge q}{\therefore\ p} \qquad \frac{p}{\therefore\ p \vee q}$$

†**3.** There are two notorious invalid arguments that look deceptively like *modus ponens* and *modus tollens*:

$$\frac{p \to q \quad q}{\therefore\ p} \qquad \frac{p \to q \quad \neg p}{\therefore\ \neg q}$$

Show that each of these arguments is invalid. Give an English example that uses each of these arguments.

†**4.** Decide whether each of the following arguments is valid. If it is valid, give a formal proof. If it is invalid, show that it is invalid by finding an appropriate assignment of truth values to propositional variables.

a) $\dfrac{p \to q \quad q \to s \quad s}{\therefore\ p}$

b) $\dfrac{p \wedge q \quad q \to (r \vee s) \quad \neg r}{\therefore\ s}$

c) $\dfrac{p \vee q \quad q \to (r \wedge s) \quad \neg p}{\therefore\ s}$

d) $\dfrac{(\neg p) \to t \quad q \to s \quad r \to q \quad \neg(q \vee t)}{\therefore\ p}$

e) $\dfrac{p \quad s \to r \quad q \vee r \quad q \to \neg p}{\therefore\ \neg s}$

f) $\dfrac{q \to t \quad p \to (t \to s) \quad p}{\therefore\ q \to s}$

†**5.** For each of the following English arguments, express the argument in terms of propositional logic and determine whether the argument is valid or invalid.

a) If it is Sunday, it rains or snows. Today, it is Sunday and it's not raining. Therefore, it must be snowing.

b) If there is herring on the pizza, Jack won't eat it. If Jack doesn't eat pizza, he gets angry. Jack is angry. Therefore, there was herring on the pizza.

c) At 8:00, Jane studies in the library or works at home. It's 8:00 and Jane is not studying in the library. So she must be working at home.